

Modelování chování

Karel Richta

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta (VŠPJ), 2024

Softwarové inženýrství, SWI
03/2024, Lekce 7

<https://moodle.vspj.cz/course/view.php?id=202893>



Analýza chování

- Začínáme zpravidla seznamem funkcí - modelem jednání – ten definuje případy užití.
- Pro každý případ užití navrhujeme podrobný scénář jednání (původce, událost, akce, participant, výstupy - reakce), diagramy aktivit, příp. diagramy datových toků (návaznosti funkcí).
- Každý takový popis případu užití představuje dekompozici na jednodušší aktivity.
- Pro netriviální aktivity definujeme popis takových aktivit opět pomocí dekompozice – rozložíme je na aktivity jednodušší.
- Pro triviální aktivity popíšeme takové aktivity (nazýváme je akce) jako minispecifikace.

Model jednání ECO-skladu

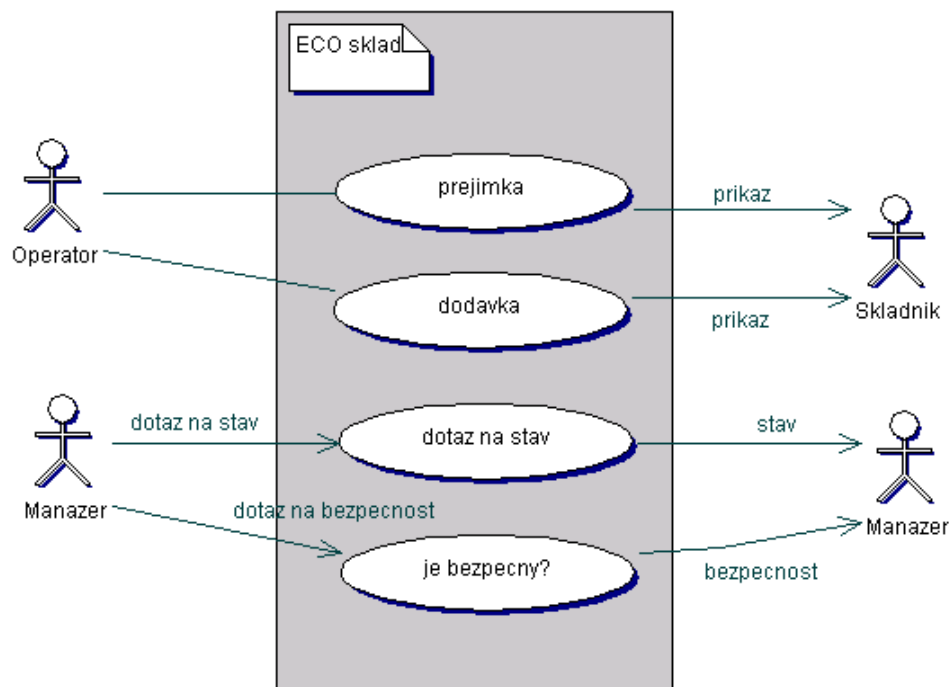
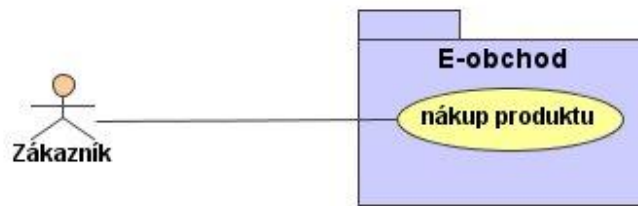


Diagram případů užití je pouhá evidence služeb, ty musí být popsány přesněji. Striktně řečeno - model jednání obsahuje diagram případů užití a jejich popis.

Jak lze služby evidované v modelu jednání popsat?

- Textovým popisem (to je podmínka nutná, nikoli postačující).
- Minispecifikací (strukturovaným popisem operace).
- Dekompozicí na služby jednodušší
 - pomocí scénářů,
 - pomocí diagramů aktivit (ve starších metodikách pomocí diagramů datových toků - DFD),
 - pomocí diagramů komunikace,
 - pomocí stavových diagramů (stavových automatů).



1. Zákazník prohlíží katalog a vybere si zboží k nákupu
2. Zákazník zvolí nákup
3. Zákazník vyplní dodací informace (adresa, expresní nebo standardní dodávka)
4. Systém zobrazí plnou cenu včetně ceny dodání
5. Zákazník vyplní platební informace (číslo kreditní karty)
6. Systém autorizuje platbu
7. Systém potvrdí prodej
8. Systém zašle potvrzovací e-mail zákazníkovi

Alternativy:

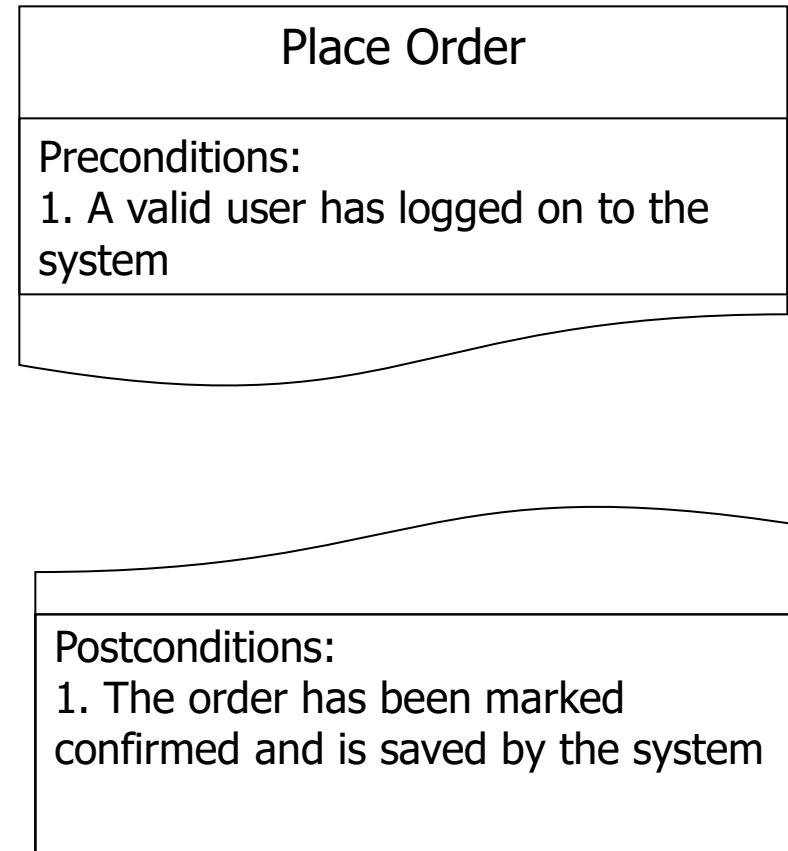
- 3a. Uživatel je pravidelným zákazníkem
 - 3a1. Systém zobrazí naposled zapamatované dodací a platební informace
 - 3a2. Uživatel může potvrdit, nebo změnit zobrazené informace a scénář pokračuje v kroku 6
- 6a. Systému se nepovedlo autorizovat platbu
 - 6a1. Zákazník může opravit platební informace, nebo zrušit nákup

Detailní specifikace případů užití

use case name		Use case: PaySalesTax
use case identifier		ID: 1
brief description		Brief description: Pay Sales Tax to the Tax Authority at the end of the business quarter.
the actors involved in the use case		Primary actors: Time
		Secondary actors: TaxAuthority
the system state before the use case can begin		Preconditions: 1. It is the end of the business quarter.
the actual steps of the use case		Main flow: / implicit time actor 1. The use case starts when it is the end of the business quarter. 2. The system determines the amount of Sales Tax owed to the Tax Authority. 3. The system sends an electronic payment to the Tax Authority.
		Postconditions: 1. The Tax Authority receives the correct amount of Sales Tax.
the system state when the use case has finished		Alternative flows: None.
alternative flows		

Předpoklady a důsledky

- Předpoklady (preconditions) a důsledky (postconditions) představují omezení.
- Předpoklady omezují stav systému před zahájením scénáře.
- Důsledky omezují stav systému po ukončení scénáře.
- Pokud nejsou žádné předpoklady ani důsledky, je třeba to zdůraznit např. klíčovým slovem "None" pod hlavičkou.



Hlavní scénář

<číslo> <někdo> <nějaká akce>

- Scénář zachycuje posloupnost kroků v rámci zpracování případu užití.
- Vždy se začíná tím že aktér něco provede (iniciuje událost).
 - Dobrá technika je zahájit scénář:
1) Případ užití se spustí, když aktér - <funkce>
- Kroky scénáře by měly být :
 - deklarativní,
 - očíslované,
 - Uspořádané v čase.
- Hlavní scénář by měl představovat úspěšné použití:
 - Vše probíhá podle očekávání, nenastaly chýby, odchylky, přerušení apod.
 - Alternativní scénáře pak popisují tyto alternativy.

Větvení: If

- Alternativy indikujeme klíčovým slovem **if**:
 - po **if** musí bezprostředně následovat výraz nabývající logické hodnoty (Boolean).
- Používejte odsazování a číslování pro vyznačení podmíněných částí.
- Pokud chcete vyznačit, co se má stát, když podmínka za **if** nabývá hodnoty false (nepravda), použijte **else** (příklad je na následujícím slidu).

Use case: ManageBasket
ID: 2
Brief description: The Customer changes the quantity of an item in the basket.
Primary actors: Customer
Secondary actors: None.
Preconditions: 1. The shopping basket contents are visible.
Main flow: 1. The use case starts when the Customer selects an item in the basket. 2. If the Customer selects "delete item" 2.1 The system removes the item from the basket. 3. If the Customer types in a new quantity 3.1 The system updates the quantity of the item in the basket.
Postconditions: None.
Alternative flows: None.

Opakování: For

- Pro opakování můžeme použít klíčové slovo **For** pro označení začátku iterace.
- Výraz, který je za klíčovým slovem **For** indikuje počet opakování vnořeného textu za příkazem **For**.

Use case: FindProduct
ID: 3
Brief description: The system finds some products based on Customer search criteria and displays them to the Customer.
Actors: Customer
Preconditions: None.
Main flow: 1. The use case starts when the Customer selects "find product". 2. The system asks the Customer for search criteria. 3. The Customer enters the requested criteria. 4. The system searches for products that match the Customer's criteria. 5. If the system finds some matching products then 5.1 For each product found 5.1.1. The system displays a thumbnail sketch of the product. 5.1.2. The system displays a summary of the product details. 5.1.3. The system displays the product price. 6. Else 6.1. The system tells the Customer that no matching products could be found.
Postconditions: None.
Alternative flows: None.

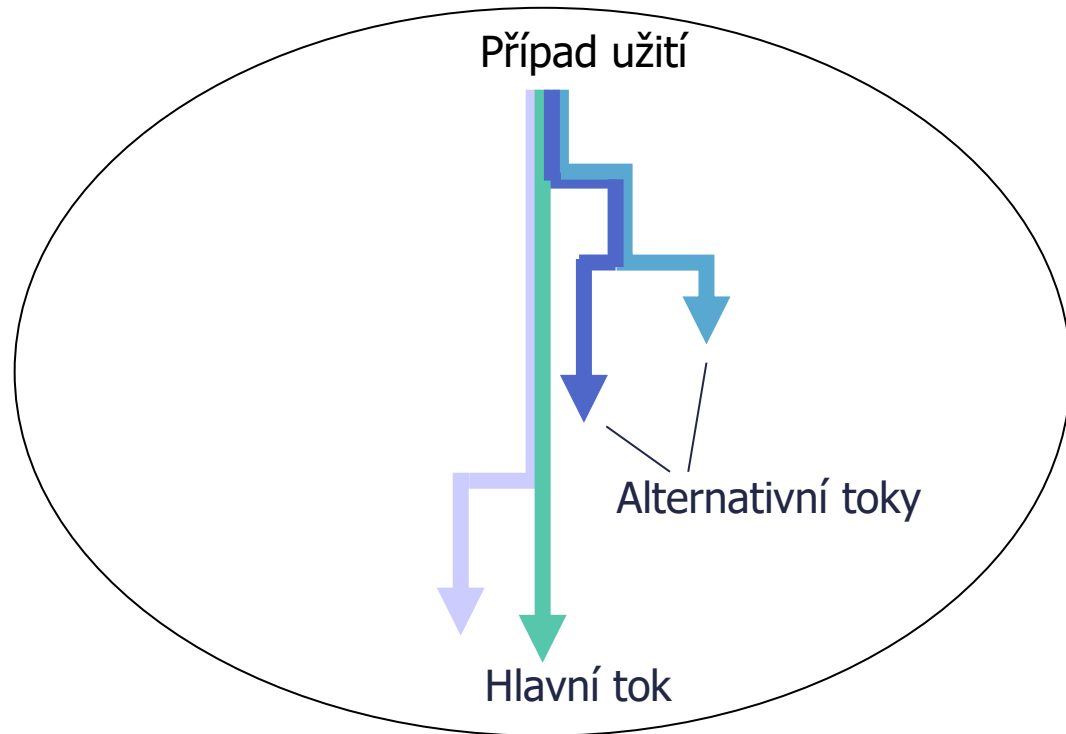
Opakování: While

- Pro opakování můžeme také použít klíčové slovo **while**, kdy je opakování dáno logickým výrazem (opakuj se, dokud má tento výraz hodnotu true).

Use case: FindProduct
ID: 3
Brief description: The system finds some products based on Customer search criteria and displays them to the Customer.
Primary actors: Customer
Secondary actors: None
Preconditions: None.
Main flow: 1. The use case starts when the Customer selects "find product". 2. The system asks the Customer for search criteria. 3. The Customer enters the requested criteria. 4. The system searches for products that match the Customer's criteria. 5. If the system finds some matching products then 5.1 while product found 5.1.1. The system displays a thumbnail sketch of the product. 5.1.2. The system displays a summary of the product details. 5.1.3. The system displays the product price. 6. Else 6.1. The system tells the Customer that no matching products could be found.
Postconditions: None.
Alternative flows: None.

Alternativy

- Alternativních toků může být více:
 - měly by zahrnovat všechny případy chybových situací, požadavků na přerušení činnosti atd,
 - alternativní toky se nikdy nevracejí do hlavního toku (main flow).
- Potenciálně existuje velmi mnoho alternativ! Musíte to nějak zvládnout:
 - vyberte nejdůležitější alternativy,
 - pokud existují skupiny podobných alternativ – dokumentujte pouze jednu z nich jako demonstrační příklad a pokud je to nezbytné, vysvětlete jak se liší.



Dokumentujte pouze takové alternativy, které jsou uvedeny v požadavcích!

Referencování alternativ

- Zapište odkaz na všechny alternativy na konec popisu případu užití.
- Hledejte alternativní toky tak, že vyzkoušíte všechny kroky v hlavním scénáři a hledáte:
 - alternativy,
 - výjimky (exceptions),
 - přerušení (interrupts).

alternative
flows

Use case: CreateNewCustomerAccount
ID: 5
Brief description: The system creates a new account for the Customer.
Primary actors: Customer
Secondary actors: None.
Preconditions: None.
Main flow: 1. The use case begins when the Customer selects "create new customer account". 2. While the Customer details are invalid 2.1. The system asks the Customer to enter his or her details comprising email address, password and password again for confirmation. 2.2 The system validates the Customer details. 3. The system creates a new account for the Customer.
Postconditions: 1. A new account has been created for the Customer.
Alternative flows: InvalidEmailAddress InvalidPassword Cancel

Příklad alternativy

notice how we
name and number
alternative flows



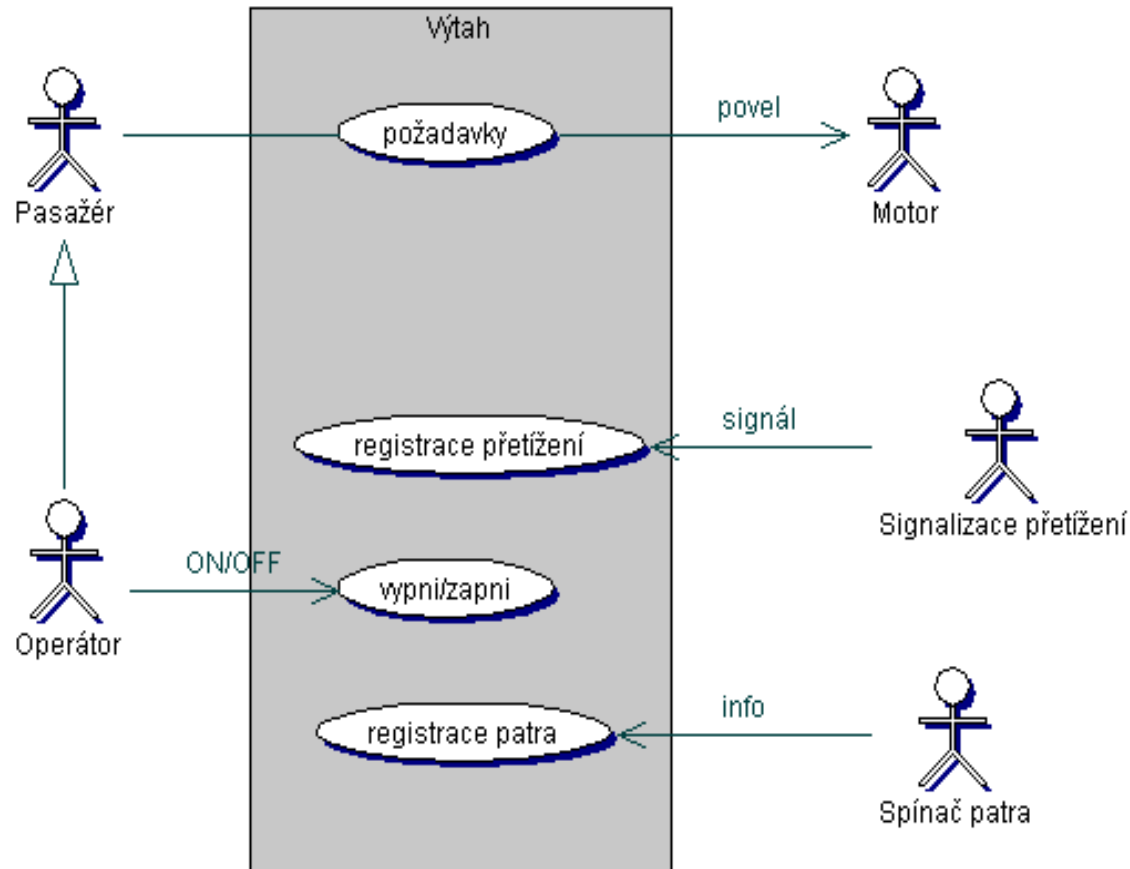
Alternative flow: CreateNewCustomerAccount:InvalidEmailAddress
ID: 5.1
Brief description: The system informs the Customer that they have entered an invalid email address.
Primary actors: Customer
Secondary actors: None.
Preconditions: 1. The Customer has entered an invalid email address
Alternative flow: 1. The alternative flow begins after step 2.2. of the main flow. 2. The system informs the Customer that he or she entered an invalid email address.
Postconditions: None.

always indicate how the
alternative flow begins.
In this case it starts after
step 2.2 in the main flow

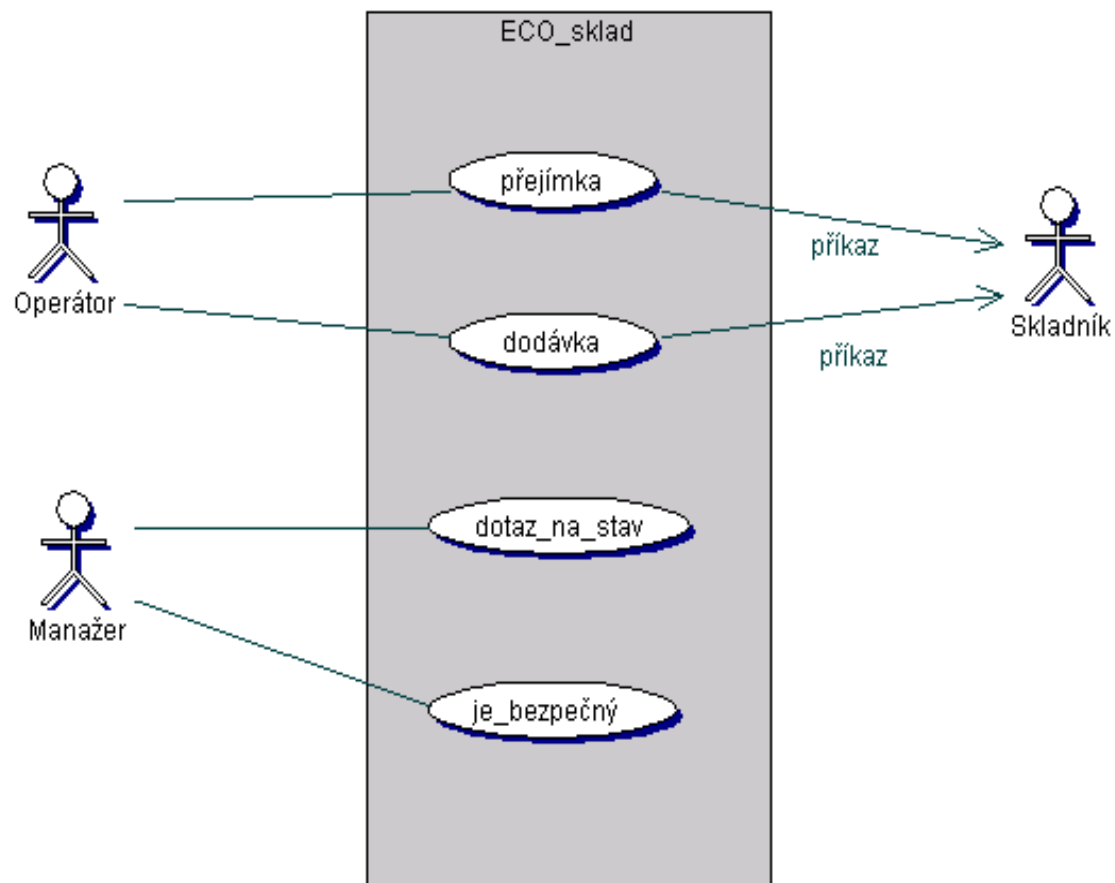


- Alternativní tok může být spuštěn místo hlavního toku – nastartován aktérem - instead of.
- Alternativní tok může být spuštěn po některém kroku – after.
- Alternativní tok může být spuštěn kdykoliv během hlavního toku - at any time.

Model jednání pro Výtah



Model jednání ECO-skladu



Trasování požadavků

- Vzhledem k tomu, že v katalogu požadavků máme zachyceny funkční požadavky a vytváříme model jednání, měli bychom je porovnat.
- Mezi požadavky a případy užití existuje vztah M..N:
 - jeden případ užití se může týkat mnoha jednotlivých funkčních požadavků,
 - jeden funkční požadavek může být realizován v mnoha případech užití.
- V ideálním případě máme CASE nástroj pro sledování požadavků:
 - označené funkční požadavky mapujeme na případy užití,
 - označené případy užití můžeme mapovat na funkční požadavky.
- Není-li CASE nástroj, můžeme vytvořit matici sledovatelnosti požadavků ručně.

		Případy užití			
Požadavky		U1	U2	U3	U4
	P1				
	P2				
	P3				
	P4				
	P5				

Matrice
sledovatelnosti
požadavků

Kdy používat analýzu případů užití?

- Případy užití popisují systém z pohledu aktérů. To je vhodné:
 - Když u systému dominují funkční požadavky.
 - Když systém poskytuje různou funkcionalitu různým aktérům.
 - Pokud má systém rozmanitá rozhraní.
- Případy užití specifikují chování z hlediska poskytovaných funkcí. Nejsou vhodné:
 - Když u systému dominují nefunkční požadavky.
 - Pokud má systém jen málo aktérů.
 - Pokud má systém jednoduchá rozhraní.

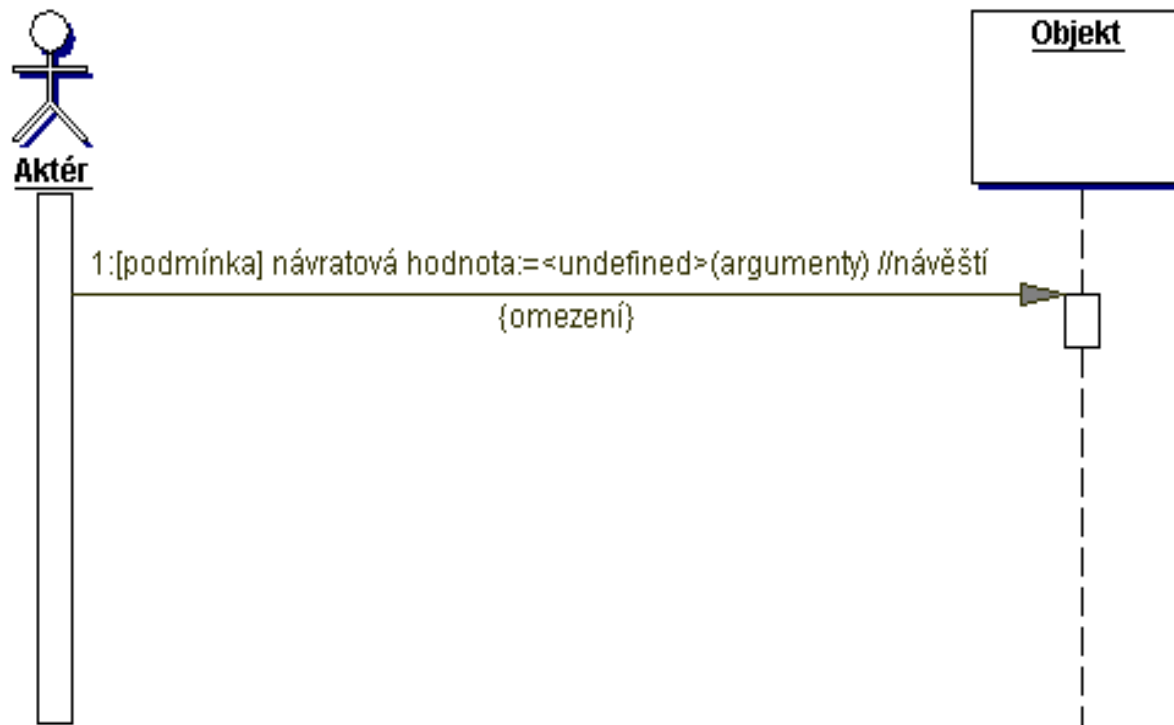
Scénáře událostí (Sequence diagrams)

(zachycení sledu událostí)

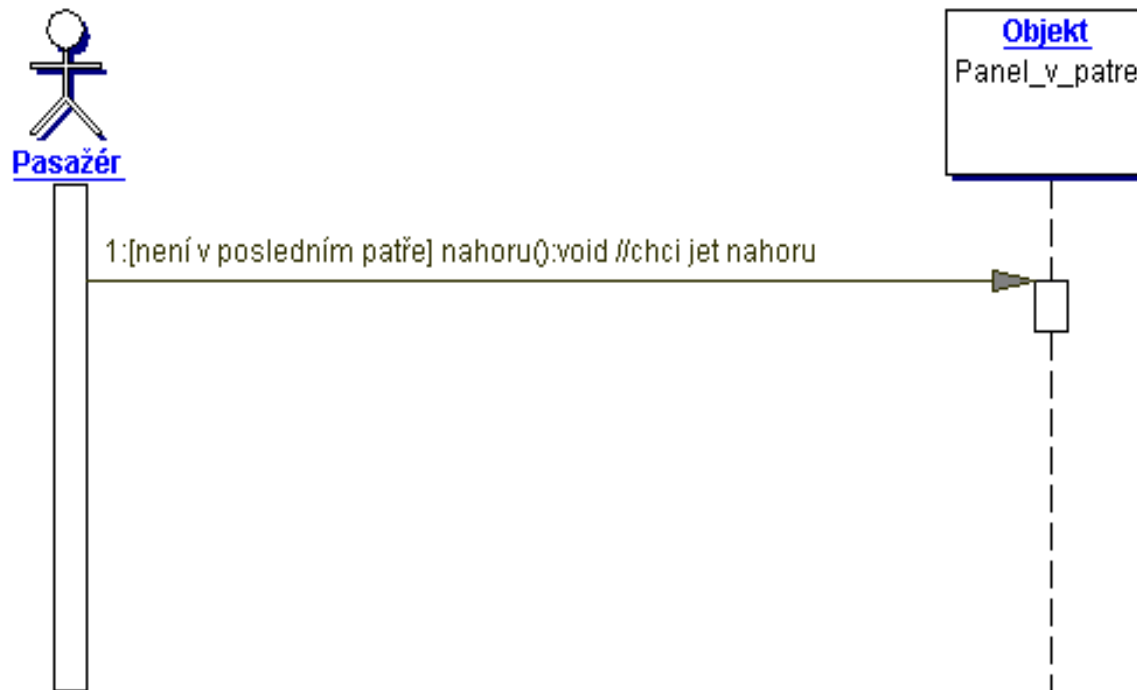
Prvky:

- **objekty** - znázorněné obvykle jako sloupce
- **interakce mezi objekty** (stimuly) - orientované šipky mezi objekty
- **události** - události, které vyvolaly interakci
- **reakce** - odezvy na události (výstupy)
- **časová osa** - pro vyznačení sledu událostí

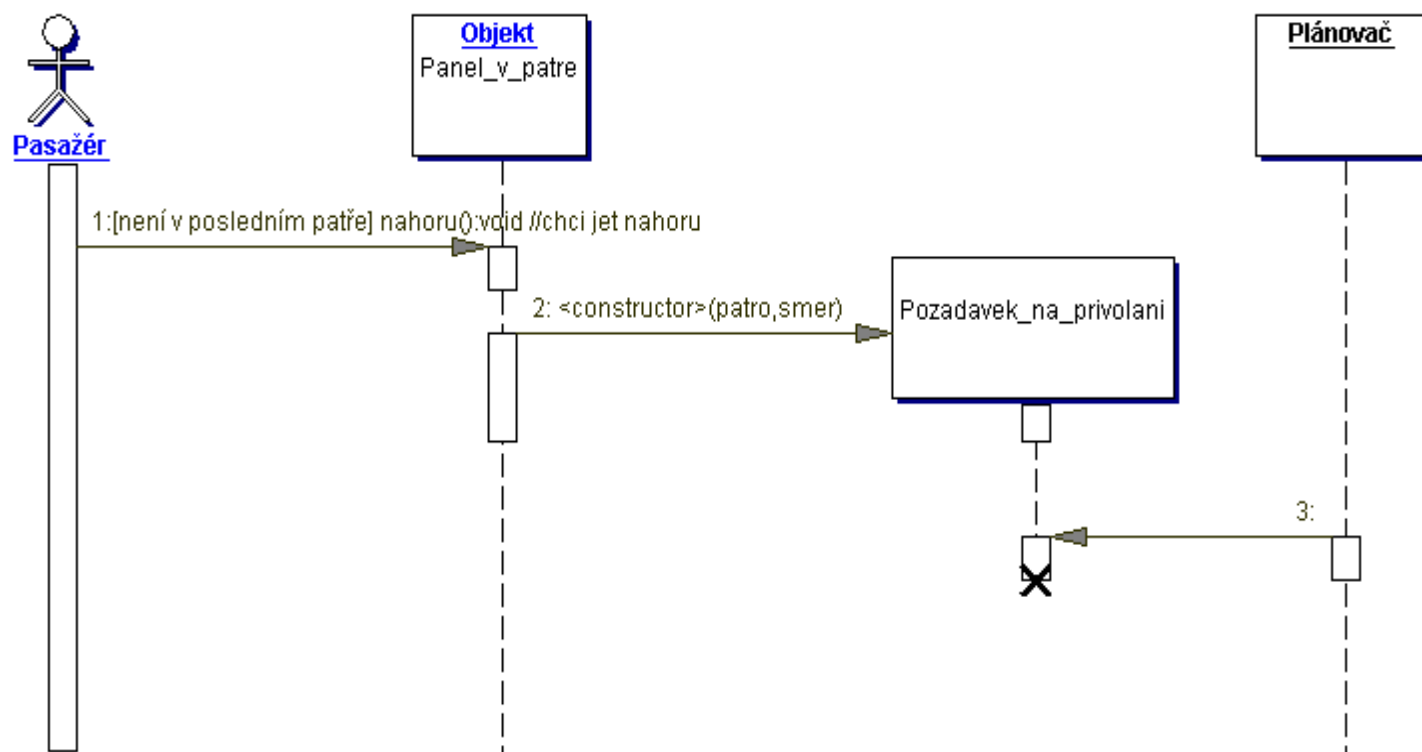
Základní princip scénáře



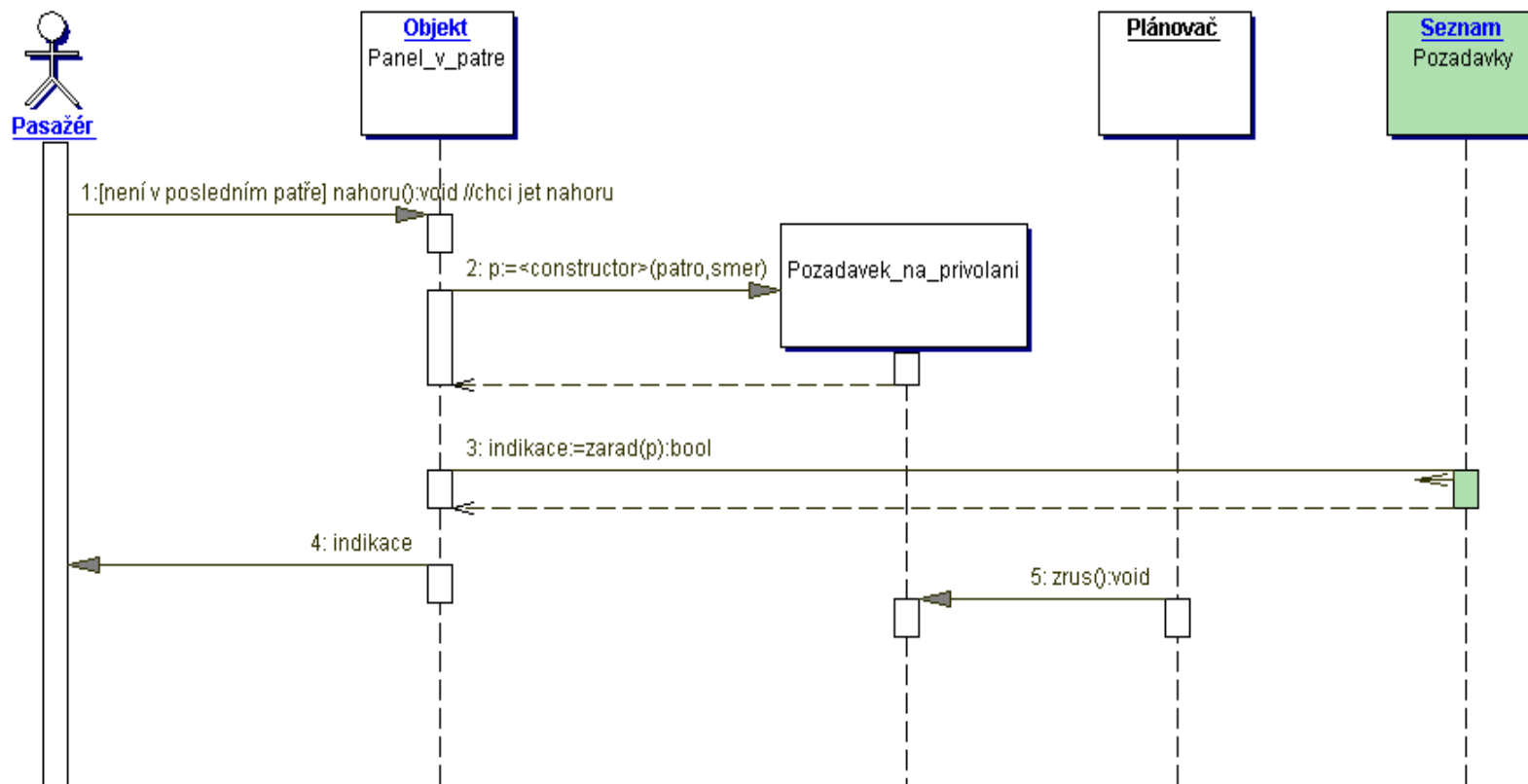
Zvolíme-li konkrétní metodu



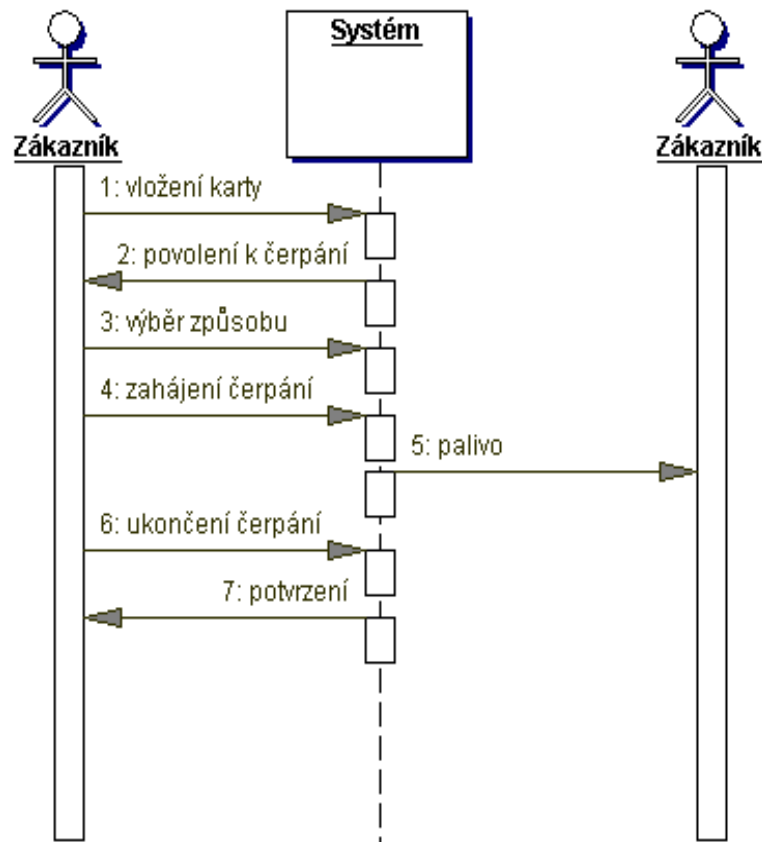
Konstrukce a destrukce



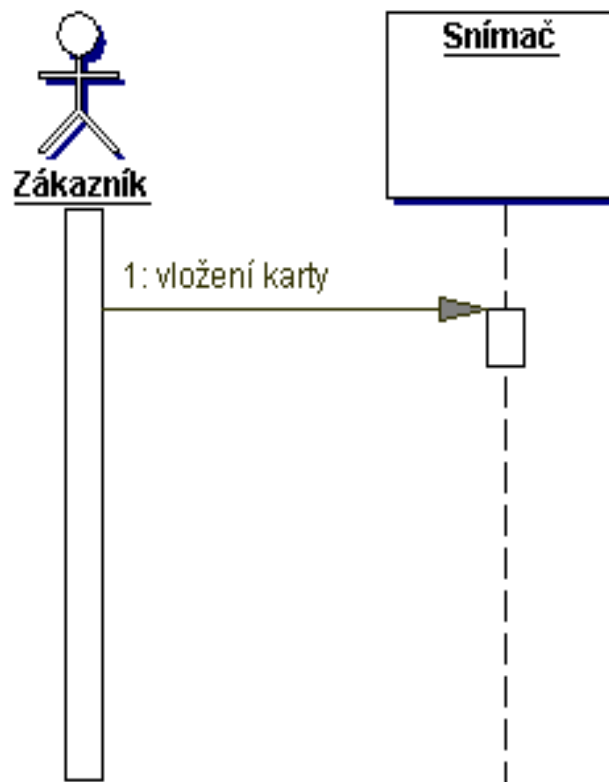
Reakce a návratové hodnoty



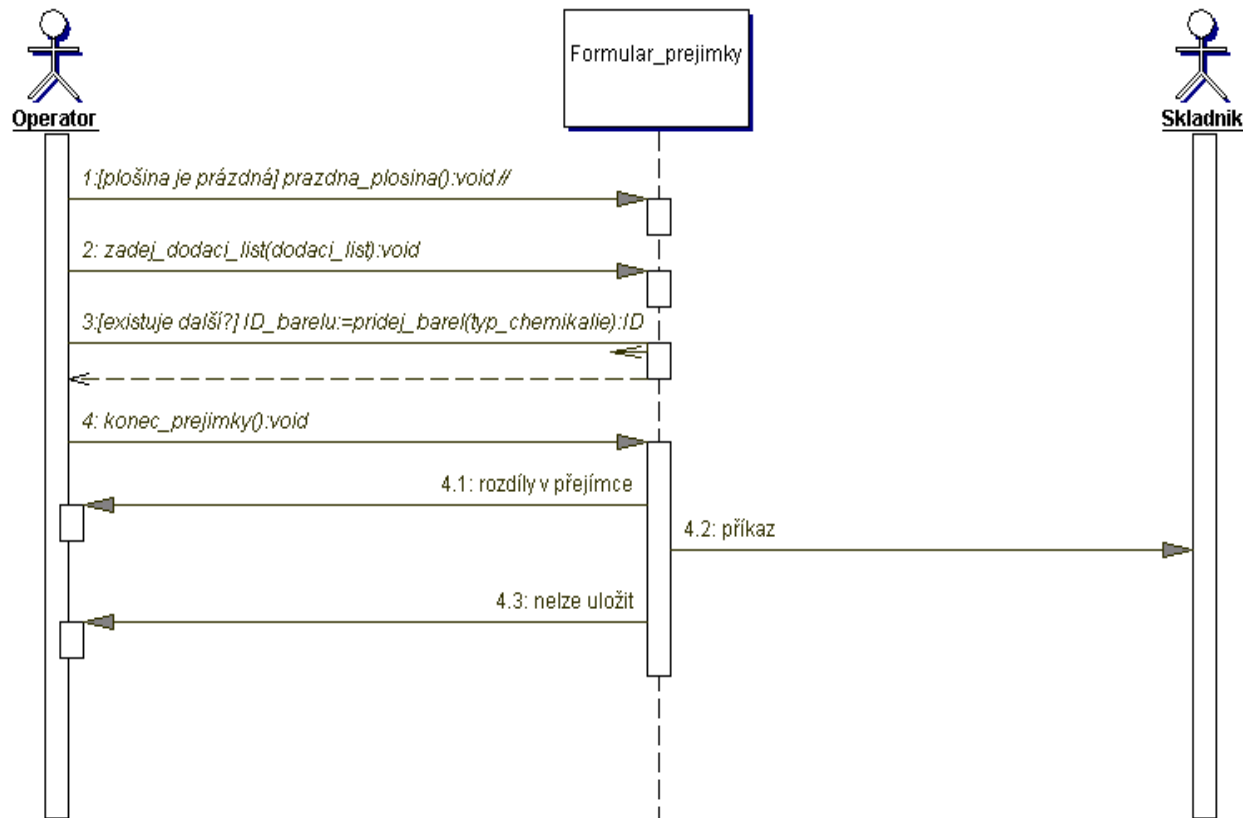
Hrubý scénář pro „čerpání“



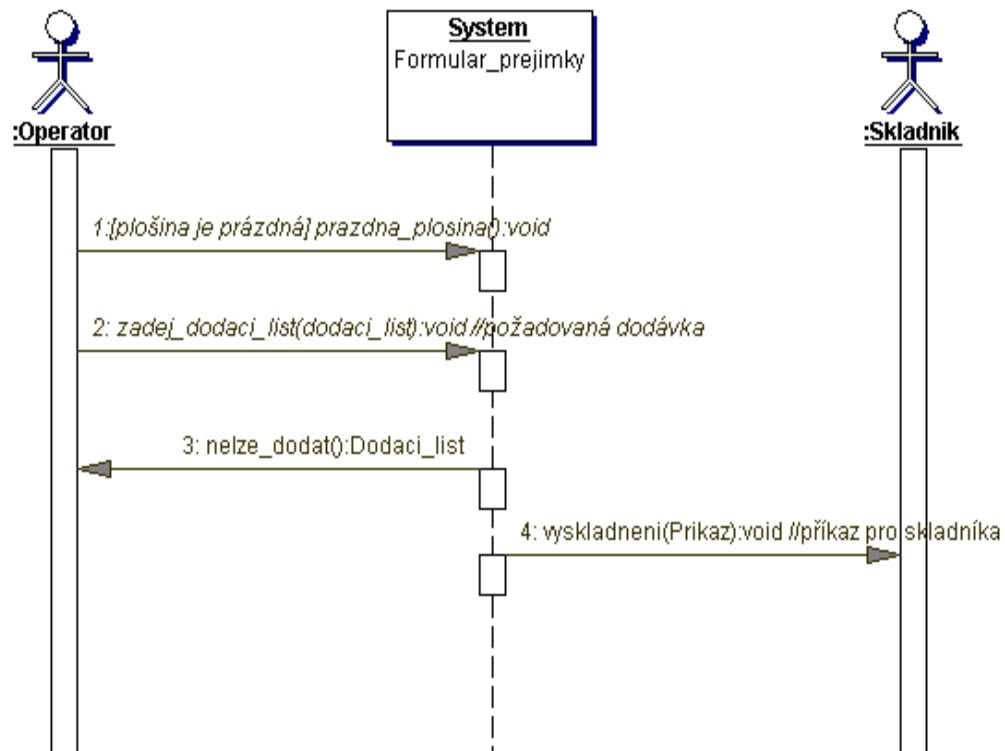
Zákazník se „autentizuje“



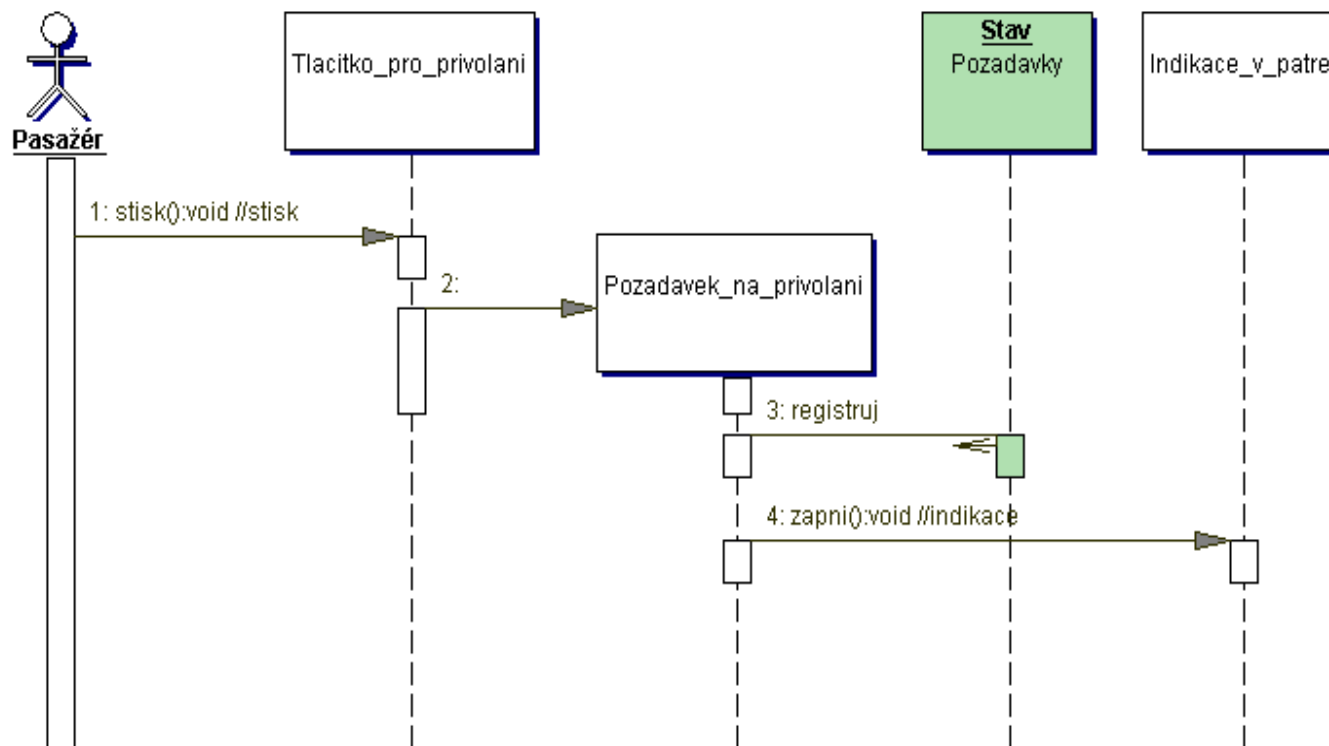
Scénář pro “přejímku”



Scénář pro “dodávku”



Scénář pro „přivolání“



Popis akce (operace, funkce)

Operation: název

Description: textový popis

Reads: jaká data jen čte

Changes: jaká data mění nebo vytváří

Sends: jaké reakce vyvolává (jaké zprávy posílá)

Assumes: co předpokládá

Results: co zajišťuje (zaručuje)

Popis pro “prázdná plošina”

Operation: prázdná plošina

Description: informuje systém, že nakládací plošina je prázdná

Reads:

Changes: plošina

Sends:

Assumes:

Results:

- vyprázdní v modelu nakládací **plošinu**
- uvolní identifikátory barelů, které jsou na **plošině**

Popis pro “zadej dodací list”

Operation: dodací list

Description: zahájí přejímku a uloží informace z dodacího listu

Reads: *supplied* dodací_list

Changes: zadaný_dodací_list

Sends:

Assumes:

Results:

- vnitřní objekt **zadaný_dodací_list** je inicializován hodnotami z fyzického **dodacího_listu**

Popis pro “barel k zařazení”

Operation: barel k zařazení

Description: každý vyložený barel je jednoznačně identifikován

Reads: *supplied* typ_chemikálie

Changes: plošina, *new* b: Barel

Sends: operátor:{ID barelu}

Assumes:

Results:

- nakládací plošina obsahuje barel b
- operátor dostane identifikaci **ID barelu**
- atribut **b.typ** je nastaven na **typ_chemikálie**
- atribut **b.ID** je nastaven na identifikaci **ID barelu**

Popis pro “konec přejímky”

Operation: konec přejímky

Description: operátor informuje systém, že již byly vyloženy všechny barely

Reads: zadaný_dodací_list

Changes: plošina, budovy ve skladu

Sends: operátor:{rozdíly v přejímce, nelze uložit},
skladník:{příkaz pro skladníka}

Assumes:

- **sklad** je bezpečný

Popis pro “konec přejímky” (pokr.)

Results:

- pro všechny barely, které lze do **skladu** umístit, přesune v modelu jejich umístění do vhodné **budovy** a vytvoří **příkaz pro skladníka(kam: alokační seznam)**
- pokud existují rozdíly mezi **zadaným_dodacím_listem** a skutečnou dodávkou, vytvoří se **rozdíly v přejímce(navíc, chybí: seznam barelů)**
- pro všechny barely, které nelze do skladu umístit vytvoří **nelze uložit(co: seznam barelů)**
- **sklad** je bezpečný

Matice CRUD

- Řádky odpovídají typům objektů.
- Sloupce odpovídají funkcím.
- V průsečíku je zapsáno zda funkce C,R,U a/nebo D odpovídající data.
- V každém řádku by mělo někde být vše (některá funkce musí objekt vytvářet, jiná využívat, či rušit).

Matrice CRUD pro ECO sklad

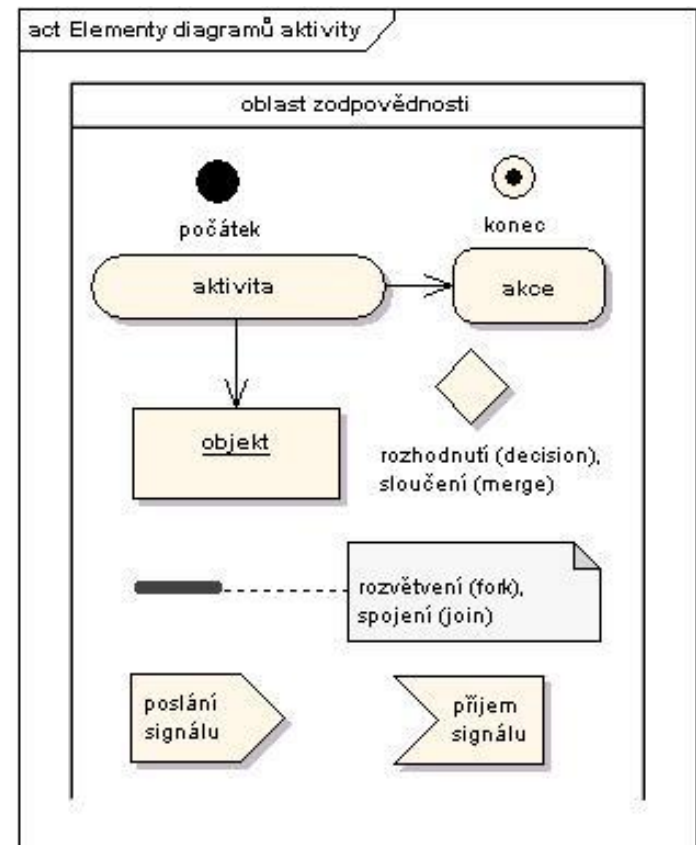
	Prázdná plošina	Zadej dodací list	Zařaď barel	Konec přejímky	Dodávka	Zahájení práce systému ECO sklad	Ukončení práce systému ECO sklad
Plošina	U		U		U	C	D
Sklad				U	U	C,Get	D,Save
Monitor				U,Print	U,Print	C	D
Barel			C				
Dodací list		C		R,D			
Příkaz				C,Print	C,Print		

Co jsme zjistili?

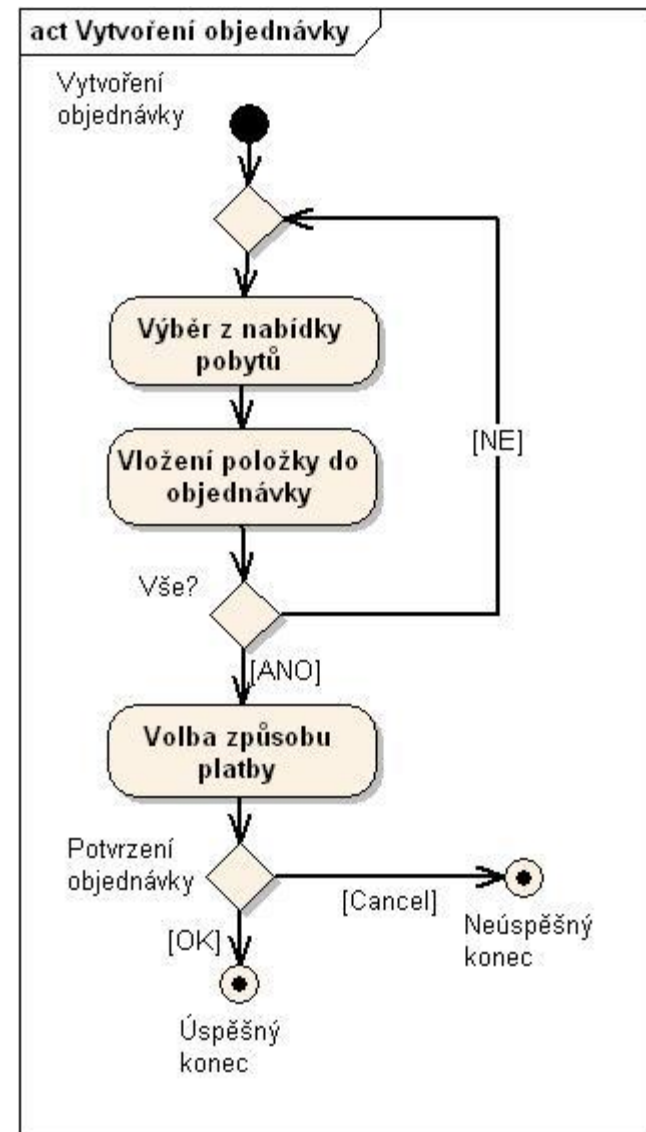
- Potřebujeme ještě v rámci nějaké funkce reprezentaci barelu zrušit.
- Mohla by to udělat funkce „dodávka“, neboť po vyskladnění barelu jeho životní cyklus končí.
- Doplníme tedy do popisu funkce dodávka požadavek „pokud v rámci dodávky využijeme některý barel, vymažeme jeho reprezentaci z obsahu skladu a zrušíme ji“.
- Do matice CRUD přidáme odpovídající D.

Diagramy aktivity

- Co všichni známe:
- Co není tak obvyklé:
- Co není známo vůbec:



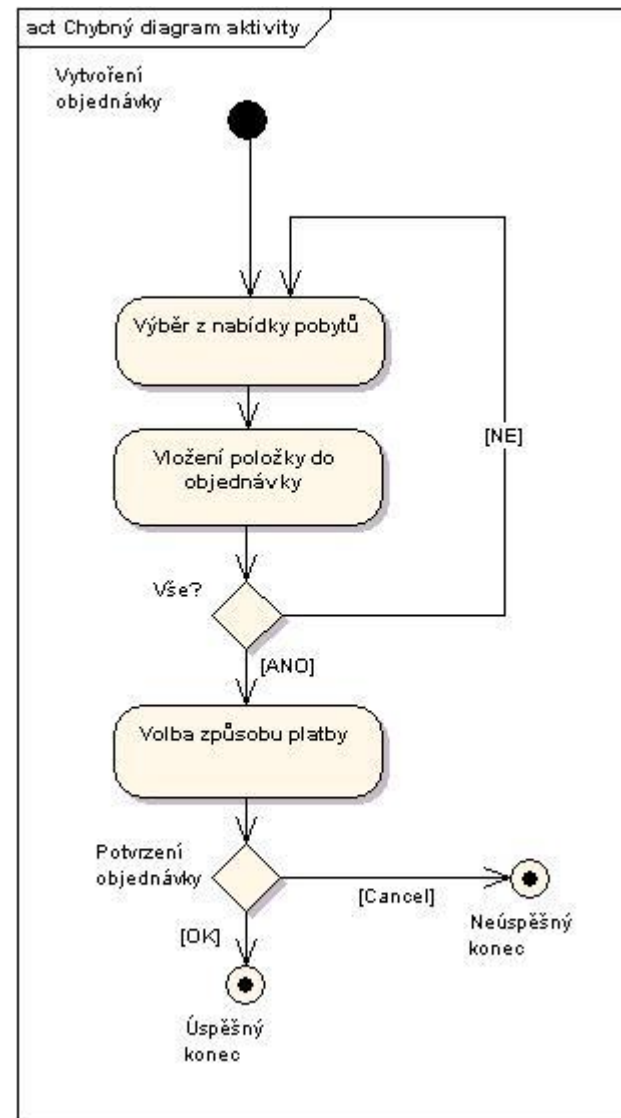
Příklad: Diagram aktivity při vytváření objednávky v HRS



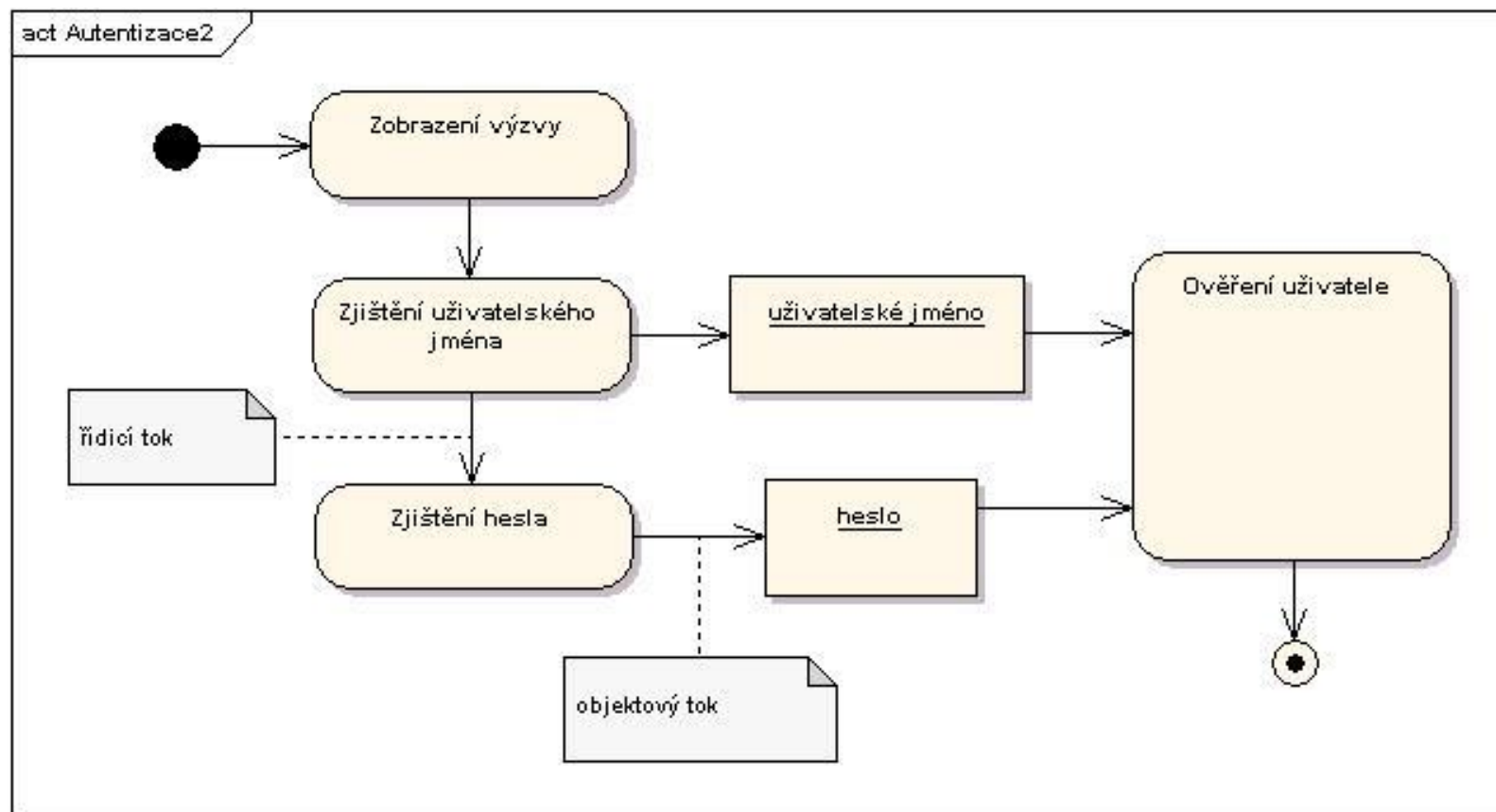
Příklad: Podobný diagram, který ale obsahuje chybu

?

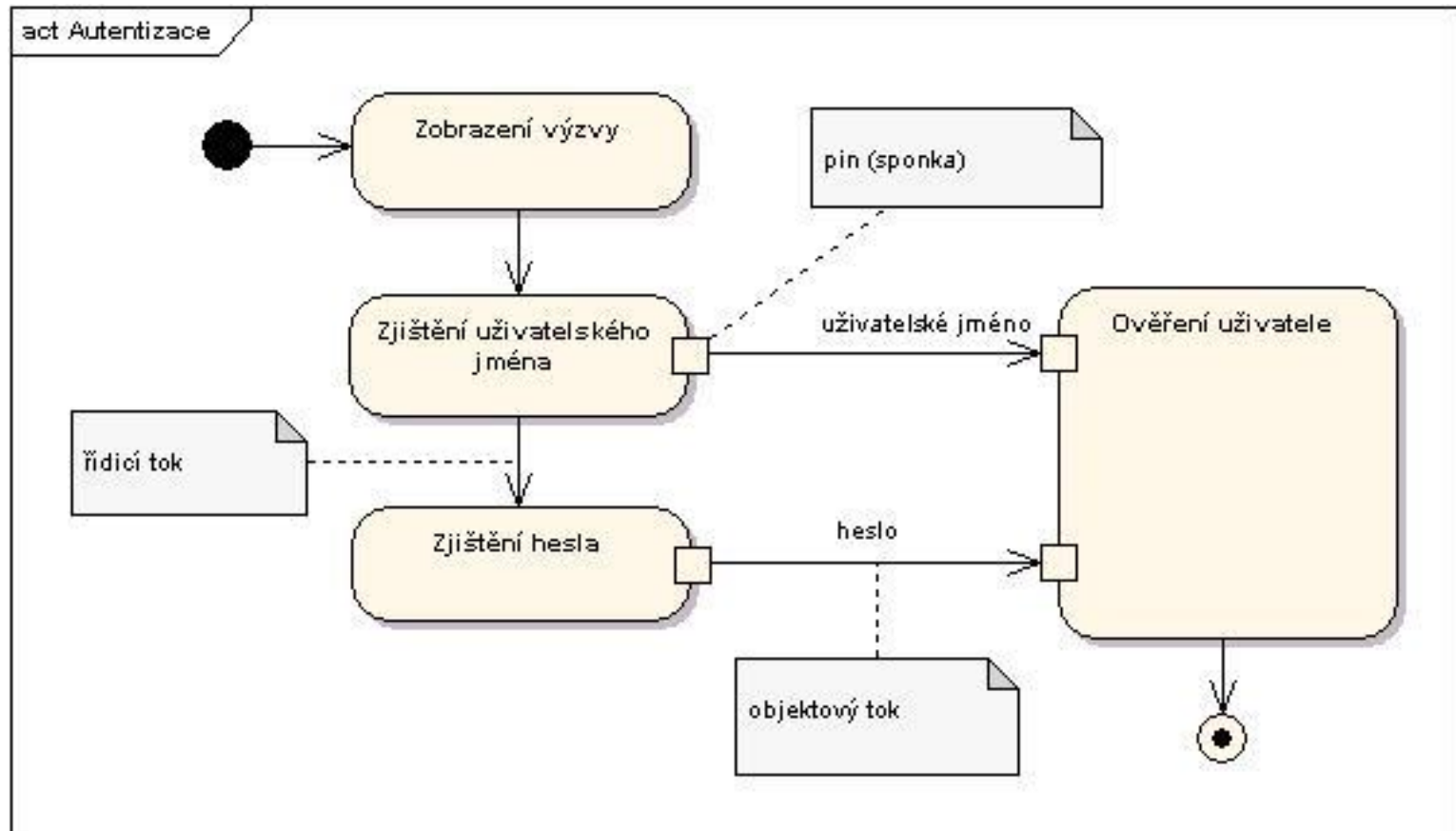
Aktivita se spustí, pokud jsou všechny vstupy aktivovány



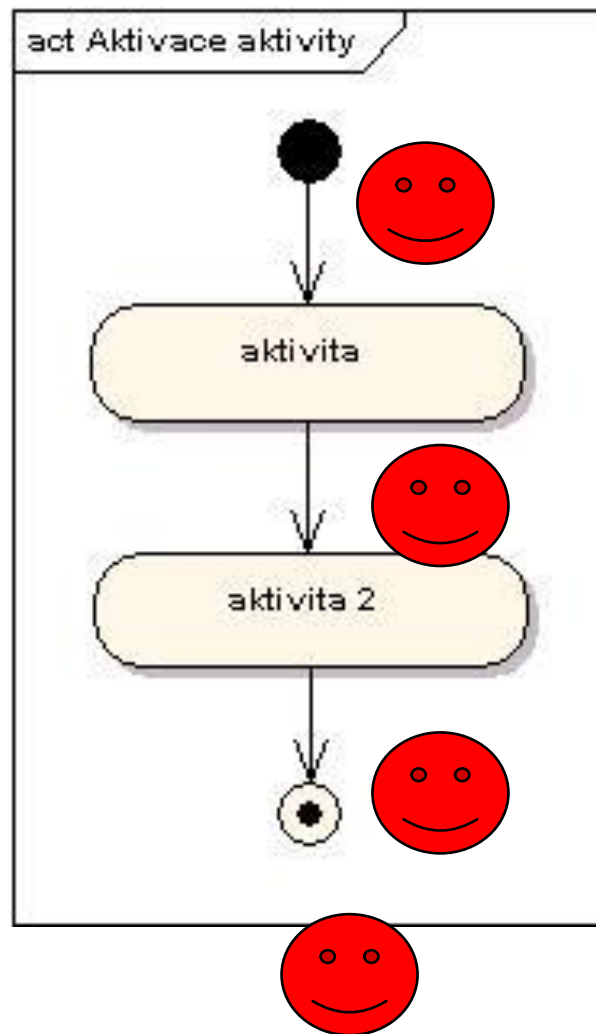
Řídicí a objektové toky



Řídicí a objektové toky jinak

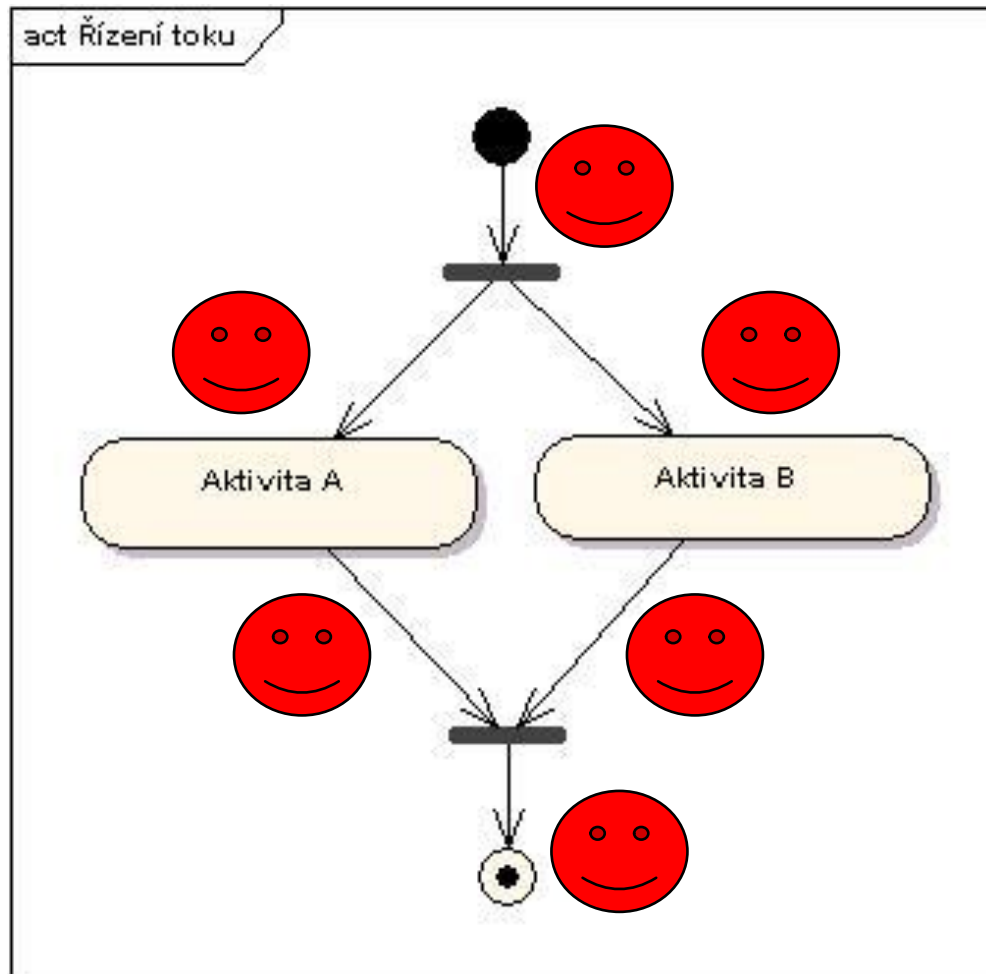


Řízení toku

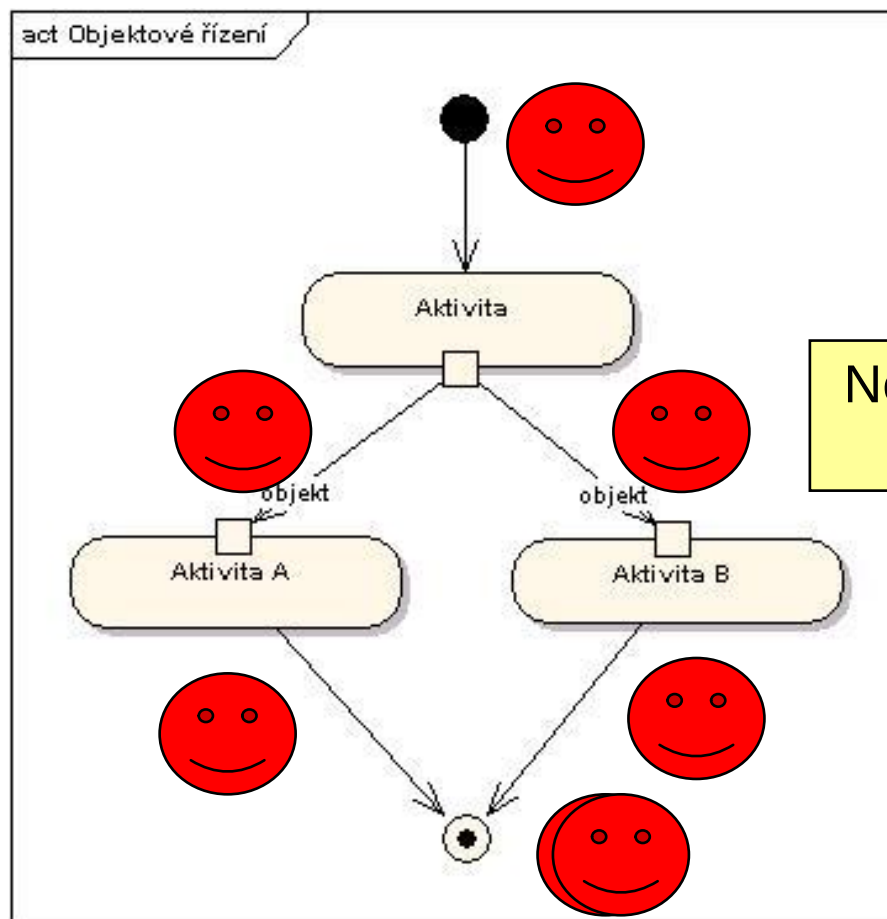


pešek
(token)

Paralelně

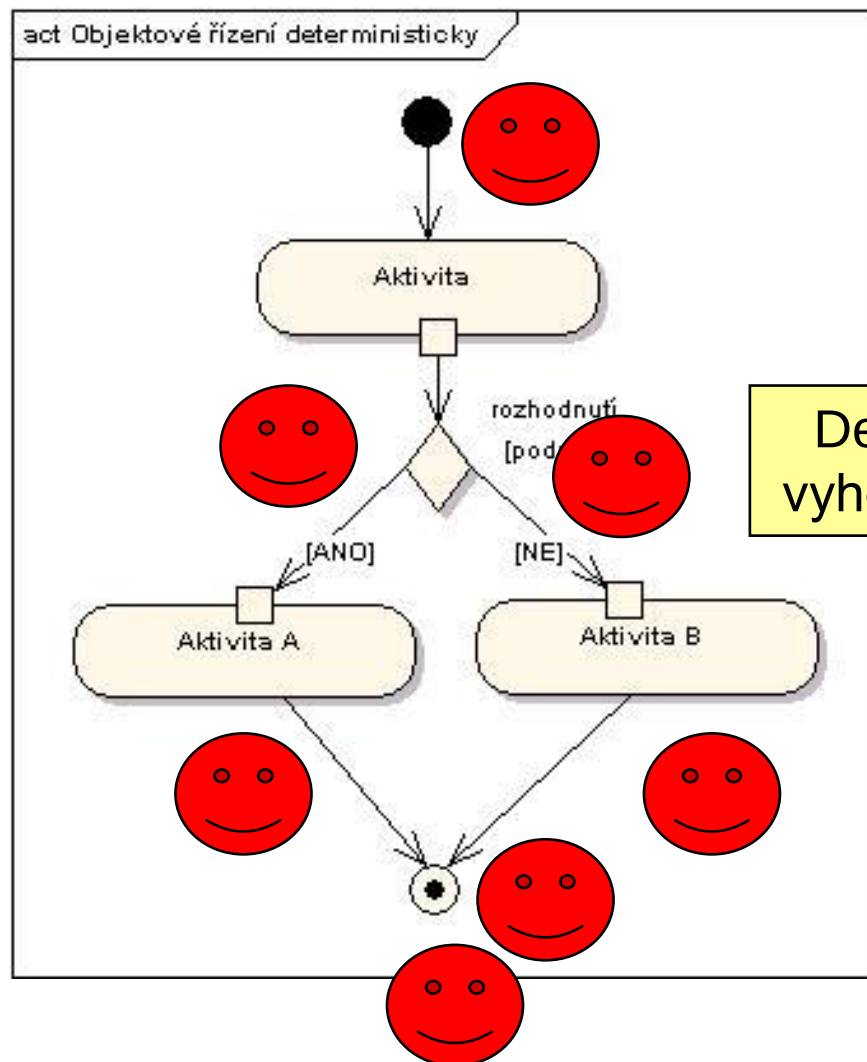


Objektové řízení

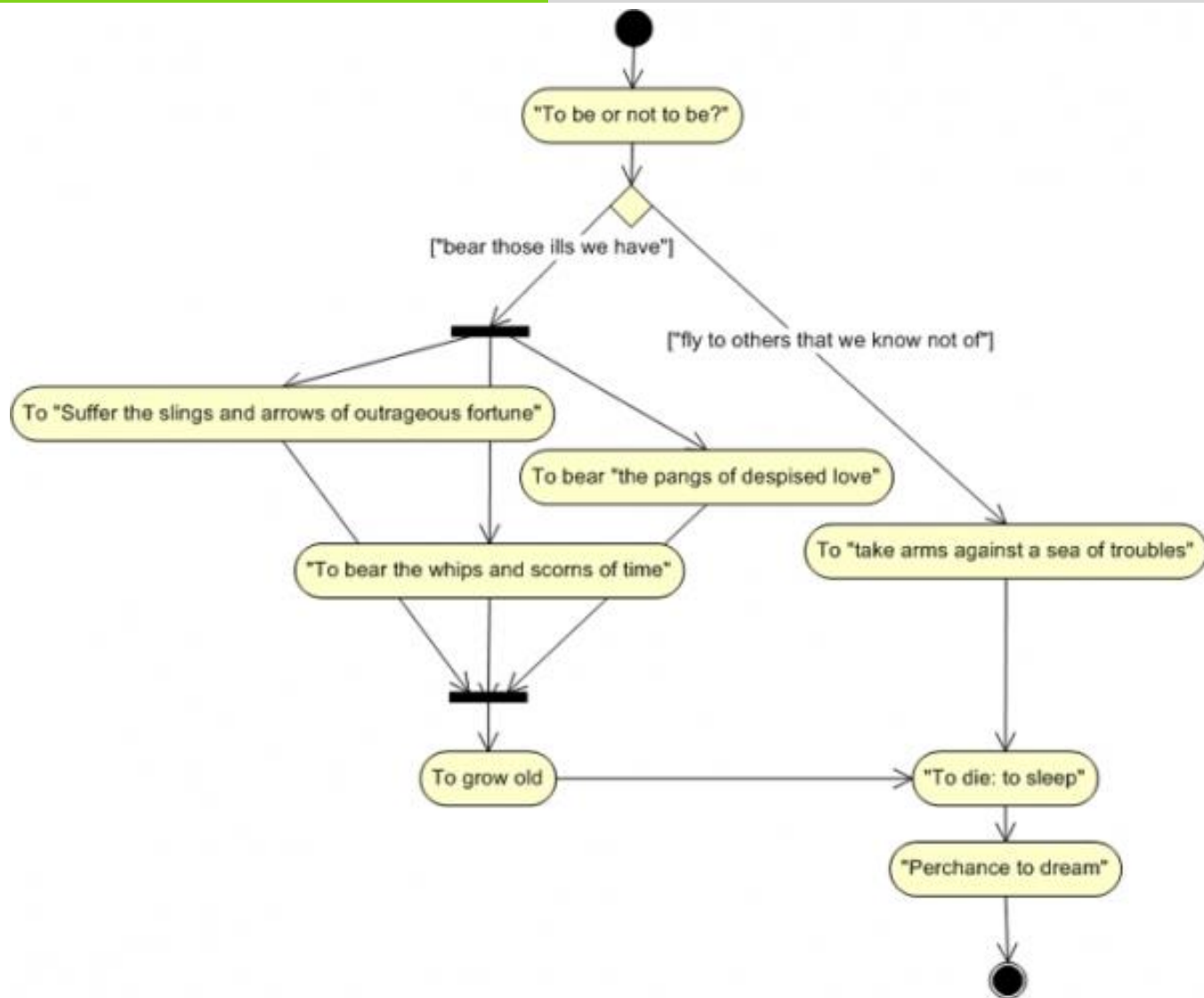


Nedeterministicky se
zvolí z možných

Deterministický



Deterministicky se vyhodnotí podmínka.



© All the text in the inverted commas is the copyrighted property of William Shakespeare

HAMLET: Být nebo nebýt – to je otázka:
je důstojnější zapřít se a snášet surovost osudu a jeho rány, anebo se
vzepřít moři trápení a skoncovat to navždy?

Zemřít, spát – a je to.

Spát – a navždy ukončit úzkost a věčné útrapy a strážně, co údělem
jsou těla – co si můžeme přát víc, po čem toužit? – Zemřít, spát – spát,
možná snít – a právě to je zrada.

Až ztichne vřava pozemského bytí, ve spánku smrti můžeme mít sny –
to proto váháme a snášíme tu dlouhou bídu, již se říká život.

Neboť kdo vydržel by kopance a výsměch doby, aroganci mocných,
průtahy soudů,

znesvěcenou lásku, nadutost úřadů a ústrky, co slušnost věčně sklízí
od lumpů, když pouhá dýka srovnala by účty, a byl by klid?

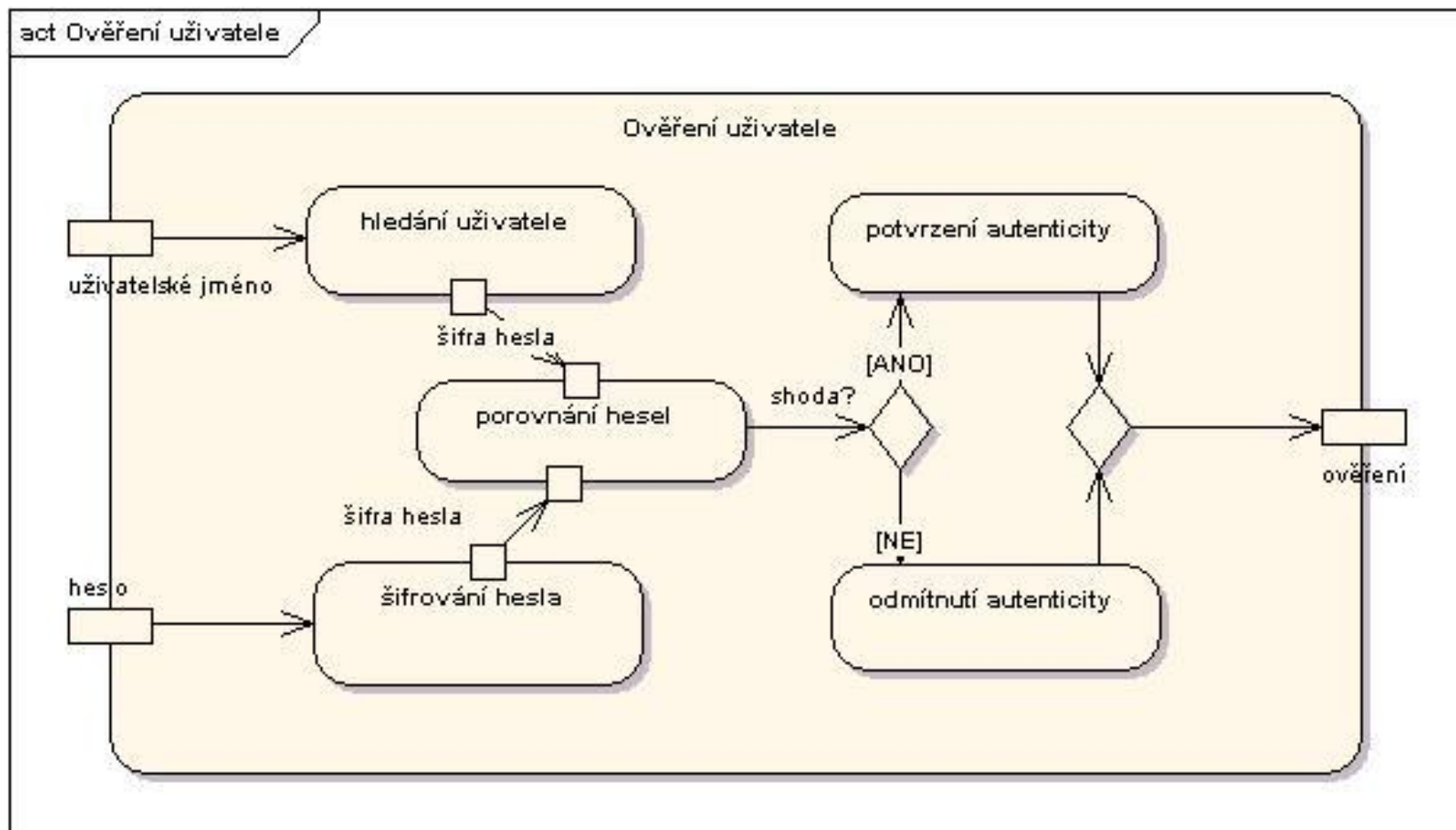
Kdo chtěl by nést to břímě, úpět a plahočit se životem, nemít strach z
toho, co je za smrtí, z neznámé krajiny, z níž poutníci se nevracejí.
To nám láme vůli – snášíme radši hrůzy, které známe, než abychom šli
vstříc těm neznámým.

Tak svědomí z nás dělá zbabělce a zdravá barva rozhodného činu se
roznemůže zbledlou meditací,
záměry velké významem a vahou se odvracejí z vytčeného směru a
neuzrají v čin.

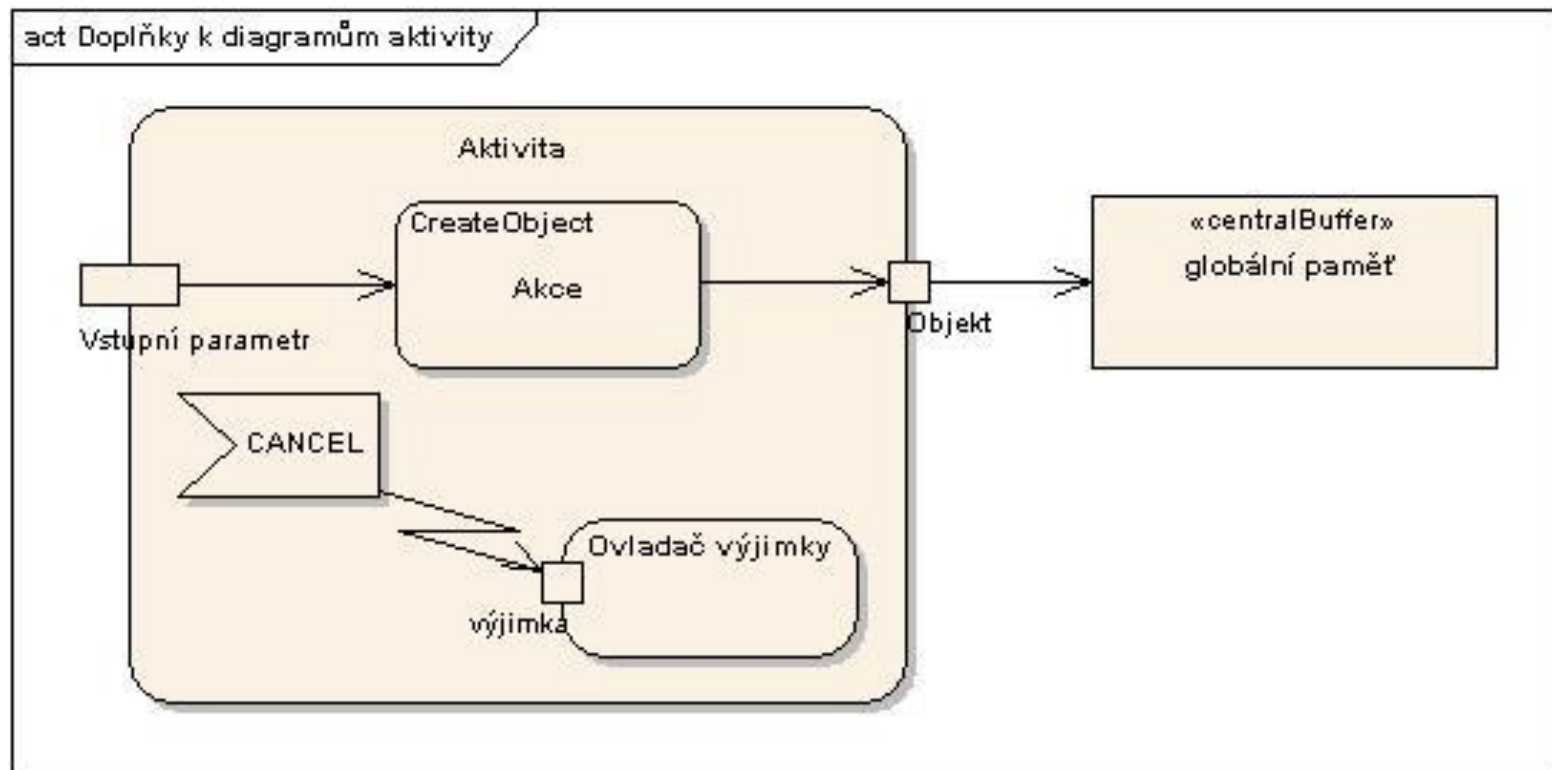
(William Shakespeare: "Hamlet" Praha Knižní klub 2002, III. 1. Str. 66,
překlad Martin Hilský)



Parametry aktivit

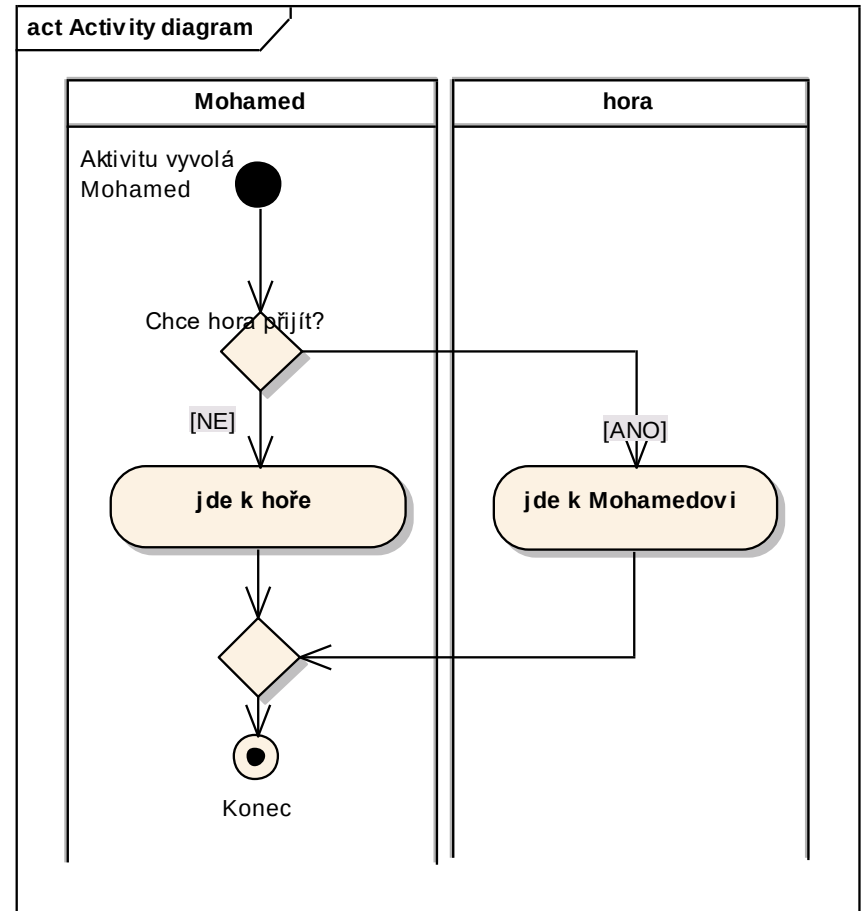


Doplňky k diagramům aktivity

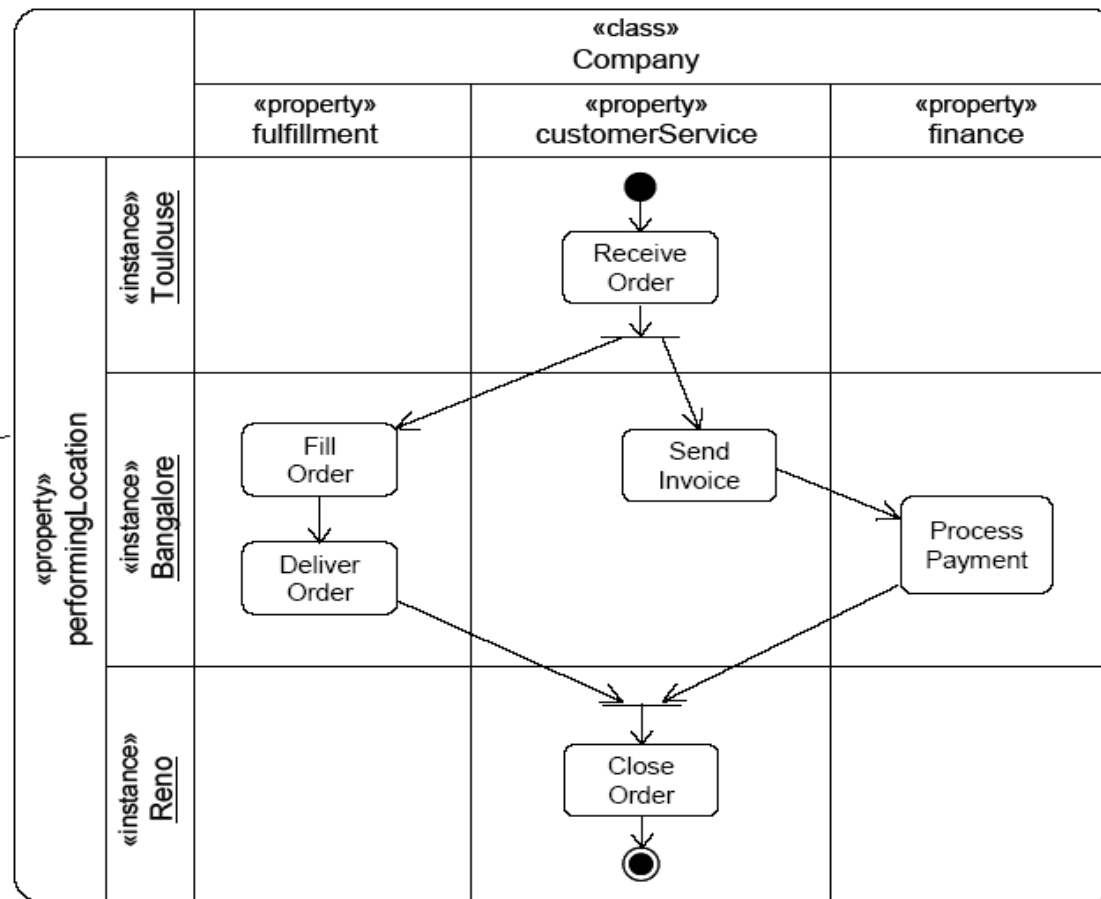


Plavecké dráhy (swimlanes)

- Oblasti zodpovědnosti.



Plavecké dráhy ve více dimenzích



Akce a aktivity

Akce

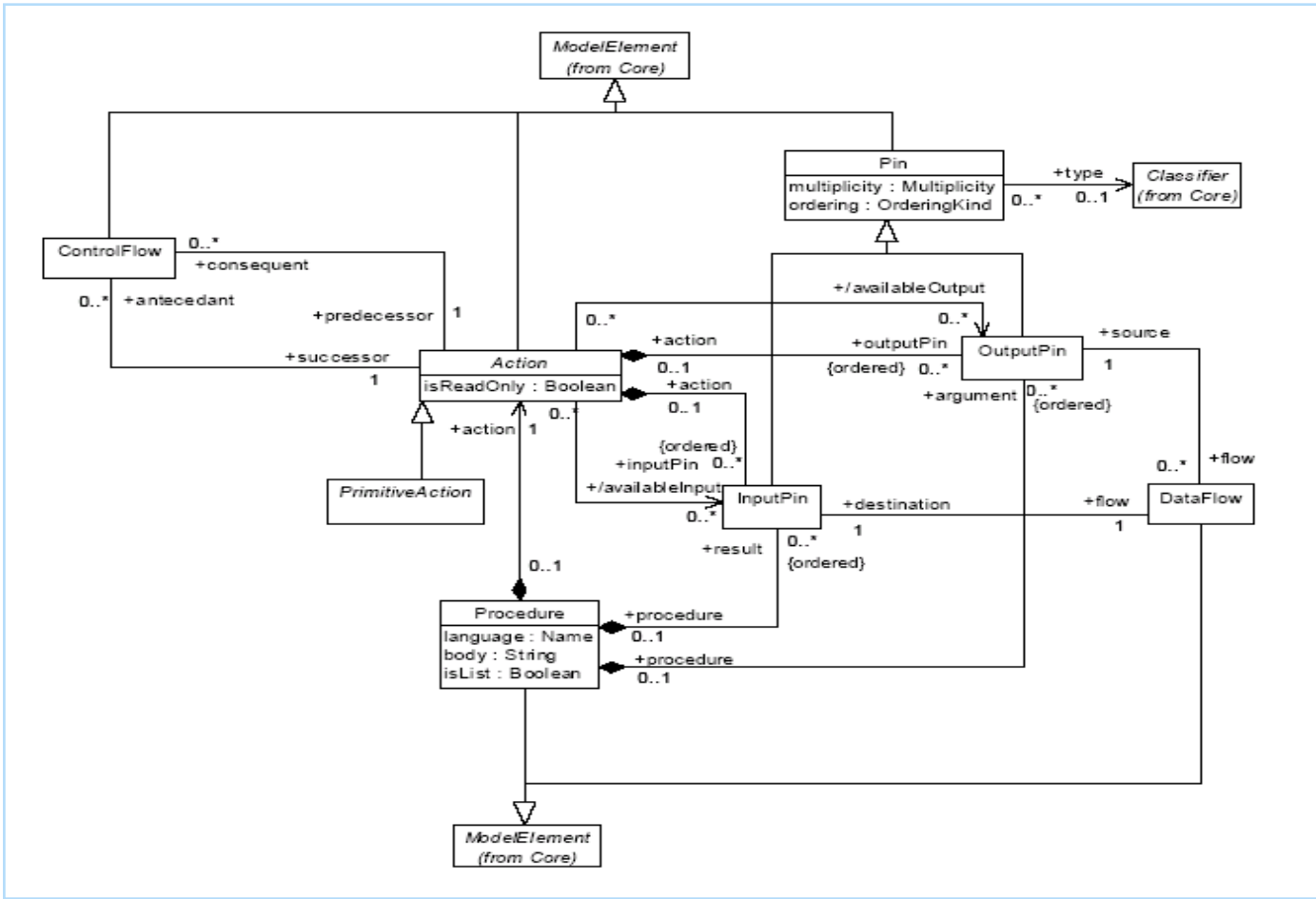
- Základní jednotka při popisu chování.
- Akce přijímá množinu vstupů a konvertuje ji na množinu výstupů.
- Některé akce modifikují stav systému, ve kterém jsou provedeny.
- Akce jsou obsaženy v aktivitách.

Akce a aktivita

UML 2 definuje více než 50 akcí

- CallOperationAction
- CallBehaviorAction
- CreateObjectAction
- DestroyObjectAction
- ReadVariableAction
- WriteVariableAction

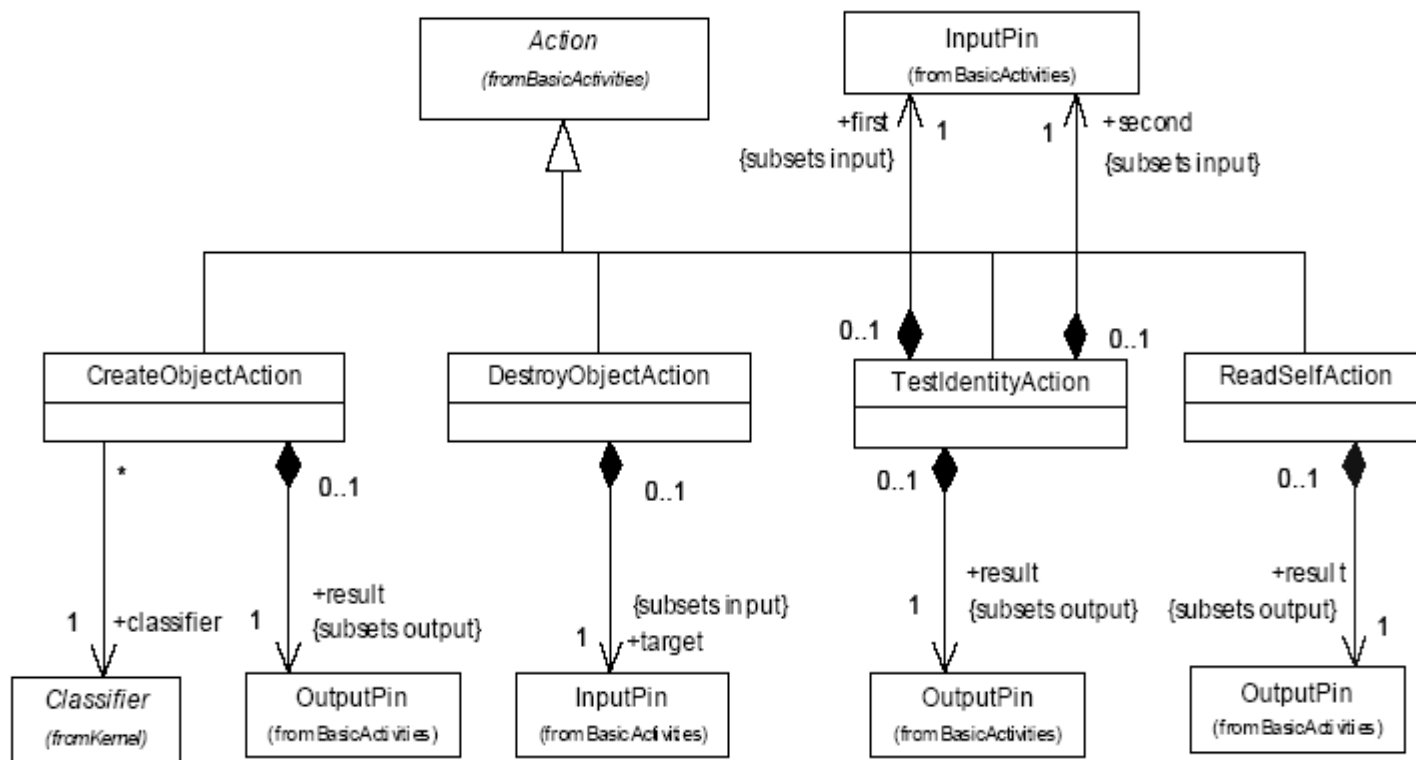
Základ akcí (metamodel)

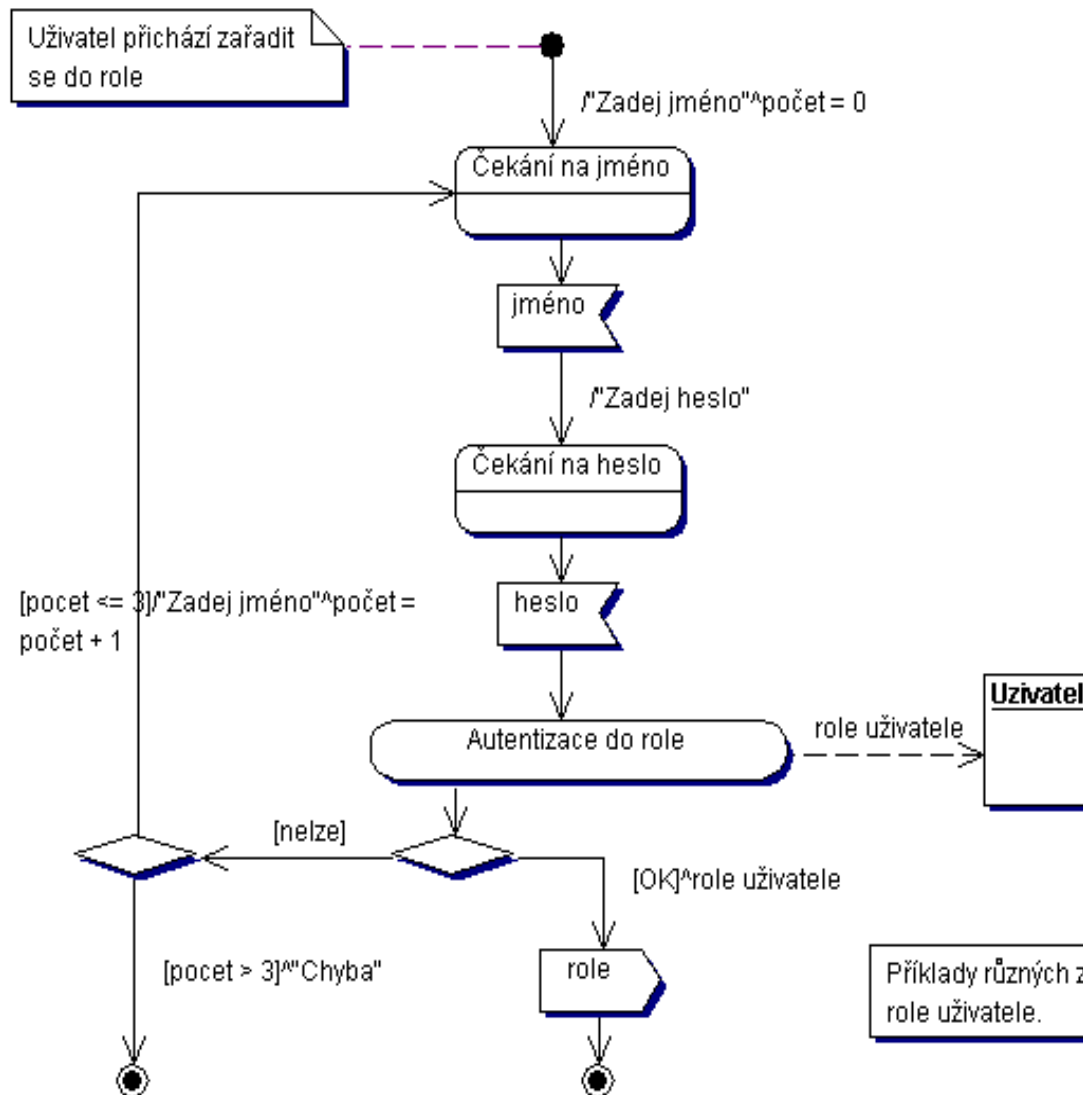


Akce a jejich význam

- Odrůda způsobu definice sémantiky založená na abstraktním stroji, který umí nějakou sadu akcí („actions“). Sémantika něčeho se pak definuje posloupností akcí, kterou to způsobí. UML 2 to do značné míry převzalo tak, že je stanovena atomická jednotka činnosti, která se nazývá „Action“. Vše ostatní se skládá z těchto elementů - např. aktivity jsou definovány jako posloupnosti akcí.
- Akce jsou klasifikovány do několika tříd - základní akce, akce vstupu a výstupu, akce vyvolání něčeho, akce pro manipulaci s objekty, akce akceptování události apod.

Př.: Akce s objekty





Kontroly analytických modelů

Výstup analýzy

Konceptuální model:

- **datový model** popisuje entity, atributy, vztahy, integritní omezení,
- **funkční model** popisuje služby, které systém poskytuje pro záznam, údržbu a využití dat,
- **dynamický model** popisuje možné stavy dat a jejich změny.

Kontrola výstupů analýzy:

- kontrola jednotlivých modelů (pohledů)
- kontrola vzájemné konzistence modelů

Kontrola datového modelu

- Je datový model úplný?
 - existuje entita pro každý typ objektu?
 - nejsou zde nadbytečné entity (entity tvořené pouze identifikací, entity s jedinou instancí, apod.)?
 - jsou zde zaneseny všechny vztahy (včetně generalizací a agregací)?
 - nejsou zde odvoditelné vztahy?
 - je model v normální formě?
 - jsou zanesena všechna integritní omezení?

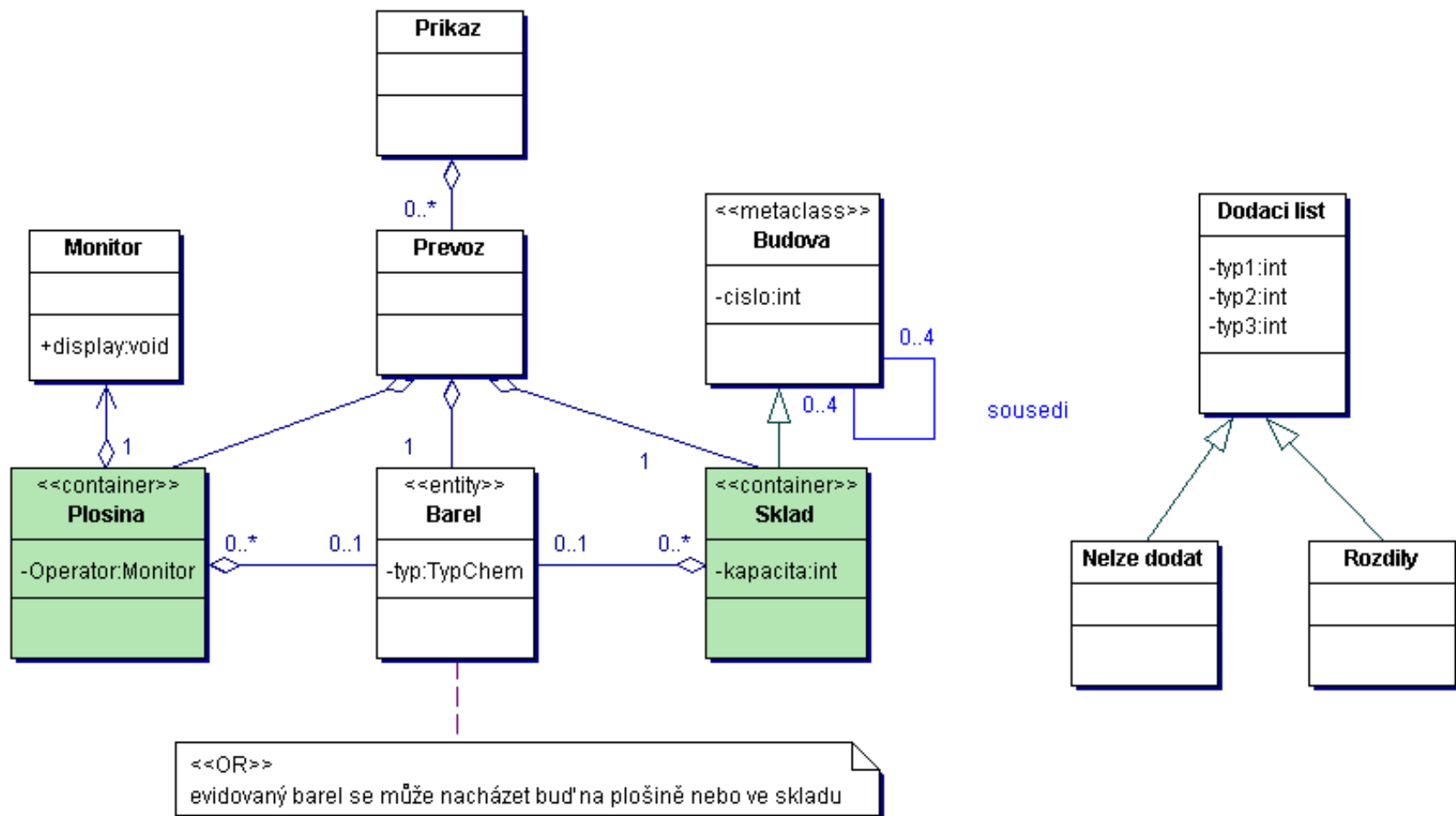
Nadbytečné entity

- entity tvořené pouze identifikací
- entity s jedinou instancí
- entity s vazbou typu 1:1
- apod.
 - dobrou technikou je představit si příklady entit a objektů?

Jsou zaneseny všechny vztahy?

- Nelze doplnit generalizace?
- Nelze doplnit agregace?
- Nelze model vylepšit?
- Příklad: Pro entitu „dodací list“ lze vymyslet pružnější model, který usnadní případné úpravy v budoucnosti

Datový model pro ECO-sklad



Nejsou zde odvoditelné vztahy?

- Zákazník si objednává zboží.
- Zákazníkovi je vystavena faktura.
- Odebrané zboží je předmětem fakturace.
- Nejsou zde odvoditelné vztahy?
- Pozn.: Odvoditelné vztahy mohou v modelu být, ale musí být jako odvoditelné předznačeny znakem „/“ a doplněny způsobem odvození (formulí, popisem v OCL).

Jsou zanesena všechna integritní omezení?

Řadu vlastností dat nelze do diagramu zanást:

- Šéf musí mít větší plat než jeho podřízení.
- V jednom skladu nelze umístit chemikálie typu „1“ a „2“.

```
context s:Sklad inv : forall(Barel x,y |  
    s.obsahuje(x) and s.obsahuje(y)  
    implies x.typ != 1 or y.typ != 2)
```

Vyvážení datového modelu

- Datový model versus datový slovník
 - každá entita, atribut a vztah v DD.
- Datový model versus funkční dekompozice
 - každá paměť a datový tok obsahuje entitu, atribut nebo vztah (nebo jejich kombinaci).
- Datový model versus minispecifikace
 - něco musí entity a vztahy vytvářet/rušit, číst/modifikovat (matice CRUD).

Kontrola funkčního modelu

- je funkční model úplný?
 - existuje funkce/metoda pro každou událost?
 - každá funkce/metoda musí být popsána dekompozicí, nebo mít minispecifikaci (vstupy a výstupy musí odpovídat)?
 - nejsou zde nadbytečné funkce/metody?

Vyvážení funkčního modelu

- Funkční model versus datový slovník
 - každá paměť a datový tok v DD,
 - každý prvek DD se někde vyskytuje (jinak je zbytečný).
- Funkční model versus datový model
 - každá data zmíněná ve funkci/metodě musí být popsána v datovém modelu.
- Funkční model versus dynamický model
 - každý řídicí proces má dynamický model (vstupy = podmínky, výstupy = akce).

Kontrola dynamického modelu

- Je dynamický model úplný?
 - existuje model pro každou entitu, která může mít různé stavy?
 - existuje model pro každý řídicí proces?
 - existuje popis životního cyklu systému?

The End